TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH

BÁO CÁO CUỐI KỲ

Môn học: Phân tích dữ liệu lớn

Phân tích dữ liệu cảm biến, dự đoán RUL và cảnh báo sớm

NASA C-MAPSS FD001–FD004

Đề tài 8: Phân tích dữ liệu IoT và bảo trì dự đoán

Học viên thực hiện:	Phan Nguyễn Mai Phương
Mã số học viên:	25002711
Ngành:	Khoa học máy tính

TP. Hồ Chí Minh, tháng 10 năm 2026

Tóm tắt

Báo cáo xây dựng và đánh giá một pipeline dự đoán Remaining Useful Life (RUL, tuổi thọ hữu ích còn lại) cho động cơ turbofan trên bộ dữ liệu mô phỏng NASA C-MAPSS. Phân tích chính được thực hiện trên subset FD001; ba subset FD002–FD004 được dùng để kiểm tra khả năng chuyển miền của mô hình.

Dữ liệu được kiểm tra schema và tính liên tục của cycle trước khi đi qua các lớp lưu trữ Bronze, Silver và Gold. Protocol lịch sử dùng trailing window 30 cycles với 126 đặc trưng và chọn HistGradientBoosting (HGB126) bằng GroupKFold 5 fold theo động cơ. Một alignment audit sau đó chấm lại HGB126 và HGB169 trên cùng 17.731 dòng, cùng khóa và cùng fold; HGB169 gồm 126 đặc trưng rolling, `cycle` và 42 đặc trưng lịch sử. Trên panel chung, HGB169 đạt MAE trung bình theo động cơ 17,0009 và RMSE 29,8068, so với 20,2143 và 33,6359 của HGB126.

HGB126 vẫn được giữ như baseline lịch sử với metric test tại điểm cắt: MAE 15,3490 cycles (khoảng tin cậy bootstrap 95%: 12,7286–18,0866), RMSE 20,5413 cycles và NASA score 1.513,2577. Báo cáo không dùng metric test của HGB169 để chọn predictor; HGB169 chỉ được đối chiếu trên panel alignment đã lưu. Với cảnh báo, threshold 30 cycles là sensitivity policy cố định: HGB126 có 51% cảnh báo đúng thời điểm, còn HGB169 có 45%; threshold 40 lần lượt là 94% và 96%. Đây là đánh giá nghiên cứu trên OOF, không phải policy bảo trì đã hiệu chuẩn.

Thử nghiệm nested cross-validation với window 10/20/30, 24 cấu hình siêu tham số và ba phương án trọng số không tìm được cấu hình mới vượt bộ tiêu chí đã chốt; similarity-based RUL pilot cũng bị dừng sau khi thua HGB ở cả ba fold. Với window 10 cycles, mô hình huấn luyện trên FD001 có MAE 47,39–60,32 cycles khi áp dụng trực tiếp lên FD002–FD004, so với 21,93–28,33 cycles khi mỗi subset tự huấn luyện và đánh giá.

Các phân tích phụ cho thấy 15/21 sensor thay đổi có ý nghĩa thống kê giữa đầu và cuối vòng đời sau hiệu chỉnh Benjamini–Hochberg; ARIMA trên Health Index có sai số dự báo thấp hơn hai baseline đơn giản trên 10 động cơ được chọn trước; ETL bằng Pandas và PySpark cho kết quả trùng khớp. Ở khía cạnh Big Data, báo cáo chứng minh một pipeline batch có tổ chức, có kiểm tra tính đúng và có thể chuyển phần ETL sang Spark; dữ liệu hiện tại vẫn vừa một máy và Spark chỉ chạy local, nên không tuyên bố đã kiểm chứng trên cluster phân tán. Các kết quả chỉ áp dụng cho dữ liệu mô phỏng C-MAPSS. Dữ liệu không có chi phí bảo trì hay quy đổi cycle sang thời gian thực, nên báo cáo không đề xuất lịch bảo trì.

Từ khóa: NASA C-MAPSS; RUL; bảo trì dự đoán; trailing window; GroupKFold; ETL; PySpark; Health Index; ARIMA.

Mục lục

Tóm tắt i

Thuật ngữ và chữ viết tắt iv

1 Giới thiệu 1
1.1 Bối cảnh và bài toán . . . 1
1.2 Mục tiêu và câu hỏi nghiên cứu . . . 1
1.3 Phạm vi và cấu trúc báo cáo . . . 1

2 Dữ liệu 2
2.1 Nguồn dữ liệu . . . 2
2.2 Cấu trúc và ý nghĩa các cột . . . 2
2.3 Đặc điểm của tập train FD001 . . . 3
2.4 Kiểm tra chất lượng và cách xử lý . . . 4

3 Cơ sở lý thuyết 6
3.1 Dự đoán RUL và các nghiên cứu liên quan . . . 6
3.2 Metric đánh giá . . . 6
3.3 Trailing window và đặc trưng thống kê . . . 7
3.4 Kiểm định ghép cặp và hiệu chỉnh đa kiểm định . . . 7
3.5 Health Index và ARIMA . . . 7
3.6 Spark, Bronze–Silver–Gold và Parquet . . . 8

4 Tổ chức, truy vấn và biến đổi dữ liệu 9
4.1 Gom dữ liệu từ nhiều file . . . 9
4.2 Lưu trữ theo lớp . . . 10
4.3 Truy vấn dữ liệu . . . 11
4.4 Các bước biến đổi . . . 12
4.5 Cột được dùng cho mô hình . . . 14
4.6 Quan hệ giữa các bảng . . . 14

5 Phương pháp 16
5.1 Tổng quan pipeline . . . 16
5.2 Huấn luyện và đánh giá mô hình RUL . . . 17
5.3 Cảnh báo sớm . . . 18
5.4 Phân tích ghép cặp đầu/cuối vòng đời . . . 18
5.5 Health Index và dự báo ARIMA . . . 19
5.6 Đánh giá chuyển miền sang FD002–FD004 . . . 19
5.7 Thử nghiệm cải thiện bằng nested cross-validation . . . 19

6 Kết quả 21
6.1 Kiểm tra dữ liệu và phân tích khám phá . . . 21
6.2 Thay đổi của sensor giữa đầu và cuối vòng đời . . . 22
6.3 Mô hình RUL . . . 23

6.4 Đối chiếu HGB126 và HGB169 trên panel chung . . . 24
6.5 Phân tích lỗi . . . 25
6.6 Cảnh báo sớm . . . 26
6.7 Thử nghiệm cải thiện bằng nested cross-validation . . . 27
6.8 Đánh giá bổ sung: đối chứng lịch sử và similarity-based RUL . . . 28
6.9 Chuyển miền sang FD002–FD004 . . . 28
6.10 Health Index và dự báo ARIMA . . . 29
6.11 ETL, parity và benchmark . . . 30

7 Thảo luận **32**
7.1 Sai số của mô hình RUL đến từ đâu . . . 32
7.2 Vì sao gần một nửa cảnh báo bị trễ . . . 33
7.3 Vì sao mô hình FD001 không chuyển được sang subset khác . . . 34
7.4 Vì sao thử nghiệm cải thiện không thành công . . . 36
7.5 Ý nghĩa đối với xử lý dữ liệu lớn . . . 36
7.6 Giới hạn . . . 37
7.7 Thí nghiệm bổ sung và hướng phát triển . . . 38

8 Kết luận **40**

Tài liệu tham khảo **41**

Thuật ngữ và chữ viết tắt

Thuật ngữ	Ý nghĩa trong báo cáo
RUL	Remaining Useful Life, tuổi thọ hữu ích còn lại: số cycle động cơ còn vận hành được trước khi hỏng.
C-MAPSS	Commercial Modular Aero-Propulsion System Simulation, bộ dữ liệu mô phỏng động cơ turbofan của NASA, gồm bốn subset FD001–FD004.
Cycle	Một chu kỳ vận hành của động cơ; là đơn vị thời gian rời rạc của dữ liệu, không quy đổi được sang giờ hoặc ngày.
Train / test	Train chứa lịch sử vận hành đến khi hỏng; test bị cắt trước khi hỏng và có file nhãn RUL riêng.
Trailing window	Đoạn w cycles liên tiếp gần nhất của một động cơ tính đến cycle đang xét; báo cáo dùng $w = 30$.
GroupKFold	Cross-validation chia fold theo nhóm; ở đây mỗi nhóm là một động cơ, nên mọi window của một động cơ nằm cùng một fold.
OOF	Out-of-fold: dự đoán cho một động cơ được tạo bởi mô hình không dùng động cơ đó khi huấn luyện.
Nested CV	Cross-validation lồng nhau: vòng trong chọn cấu hình, vòng ngoài đánh giá cả quy trình chọn.
MAE / RMSE	Sai số tuyệt đối trung bình / căn bậc hai của sai số bình phương trung bình, tính bằng cycle.
NASA score	Điểm phạt bất đối xứng, phạt dự đoán trễ (cao hơn RUL thật) nặng hơn dự đoán sớm.
CI	Confidence interval, khoảng tin cậy.
HGB	HistGradientBoostingRegressor của scikit-learn.
Data leakage	Rò rỉ dữ liệu: thông tin không có tại thời điểm dự đoán (như nhãn test hoặc cycle tương lai) lọt vào quá trình huấn luyện hoặc chọn mô hình.
EDA	Exploratory Data Analysis, phân tích khám phá dữ liệu.
ETL	Extract, Transform, Load: đọc, biến đổi và ghi dữ liệu.
Bronze / Silver / Gold	Ba lớp lưu trữ: dữ liệu gốc; dữ liệu đã kiểm tra và chuẩn hóa kiểu; bảng tổng hợp phục vụ phân tích.
Parquet / SQLite	Định dạng lưu trữ dạng cột / cơ sở dữ liệu quan hệ dạng file.
Parity	Đối chiếu kết quả của hai implementation (Pandas và PySpark) trên cùng input.
Lead time	Thời gian báo trước: số cycle từ cảnh báo đầu tiên đến khi động cơ hỏng.
Health Index	Chỉ số tổng hợp trạng thái động cơ, ở đây là thành phần chính thứ nhất của 21 sensor sau chuẩn hóa.
PCA	Principal Component Analysis, phân tích thành phần chính.

Tiếp tục ở trang sau

Thuật ngữ	Ý nghĩa trong báo cáo
ARIMA	Mô hình chuỗi thời gian tự hồi quy, sai phân và trung bình trượt.
AIC	Akaike Information Criterion, tiêu chí chọn bậc mô hình.
ADF	Kiểm định Augmented Dickey–Fuller về nghiệm đơn vị (tính dừng).
Ljung–Box	Kiểm định tự tương quan của phần dư.
Wilcoxon	Kiểm định dấu-hạng Wilcoxon cho dữ liệu ghép cặp.
BH / FDR	Hiệu chỉnh Benjamini–Hochberg nhằm kiểm soát False Discovery Rate (tỷ lệ phát hiện sai).

Danh mục hình

1 Luồng dữ liệu C-MAPSS . . . 9
2 Minh họa window và nhãn . . . 13
3 Quan hệ giữa các bảng . . . 15
4 Pipeline xử lý dữ liệu . . . 16
5 Nested cross-validation . . . 20
6 Phân phối tuổi thọ train FD001 . . . 21
7 Tương quan sensor–RUL . . . 22
8 Ảnh hưởng đầu/cuối sensor . . . 23
9 RUL thật và dự đoán test . . . 24
10 Sai số theo RUL test . . . 25
11 Quỹ đạo cảnh báo HGB126 . . . 26
12 Chênh lệch MAE RUL thấp . . . 27
13 MAE chuyển miền . . . 29
14 Dự báo Health Index bằng ARIMA . . . 30
15 RUL OOF so với thật . . . 32
16 Health Index của 10 động cơ . . . 33
17 Cảnh báo theo tuổi thọ . . . 34
18 Dịch chuyển phân phối sensor . . . 35

Danh mục bảng

2 Quy mô C-MAPSS . . . 2
3 Từ điển dữ liệu FD001 . . . 3
4 Nhóm cột train FD001 . . . 3
5 Hồ sơ sensor FD001 . . . 4
6 Chất lượng dữ liệu . . . 5
7 Vị trí nghiên cứu . . . 6
8 Các bảng FD001 . . . 10
9 Các bước biến đổi FD001 . . . 12
10 Vai trò nhóm cột . . . 14
11 Kiểm tra quan hệ bảng . . . 15
12 Công cụ và đầu ra . . . 17
13 Tuổi thọ train FD001 . . . 21
14 Thay đổi sensor . . . 22
15 So sánh mô hình GroupKFold . . . 23
16 Metric test lịch sử HGB126 . . . 24
17 Đối chiếu HGB126 và HGB169 . . . 24
18 Năm lỗi lớn nhất . . . 25
19 Phân phối lead time . . . 26

20 Độ nhạy policy cảnh báo 26
21 Kết quả vòng ngoài 27
22 Đánh đổi trong inner CV 28
23 Đánh giá chuyển miền 29
24 Kết quả dự báo Health Index 30
25 Benchmark ETL bằng Pandas 31
26 Sai số OOF theo RUL 32
27 Chẩn đoán chuyển miền 35
28 Phạm vi Big Data theo 5V 37
29 Nested CV bổ sung 38
30 Benchmark local 38

1 Giới thiệu

1.1 Bối cảnh và bài toán

Trong bảo trì dự đoán, một câu hỏi trung tâm là thiết bị còn vận hành được bao lâu trước khi hỏng. Với động cơ turbofan, đại lượng này được gọi là Remaining Useful Life (RUL). Nếu ước lượng được RUL từ dữ liệu cảm biến, đội vận hành có thể lên kế hoạch can thiệp trước khi sự cố xảy ra thay vì bảo trì theo lịch cố định hoặc sửa chữa sau khi hỏng.

Báo cáo xem dự đoán RUL là bài toán hồi quy có giám sát trên chuỗi thời gian cảm biến. Tại cycle t, mô hình chỉ được dùng dữ liệu đã quan sát đến cycle t; biến mục tiêu là số cycle còn lại trước khi động cơ hỏng. Trong tập train, mỗi động cơ được theo dõi đến khi hỏng, nên nhãn RUL được tính trực tiếp:

$$y_{i,t} = T_i - t, \qquad 1 \le t \le T_i, \tag{1}$$

trong đó T_i là cycle cuối cùng (cycle hỏng) của động cơ i và $y_{i,t}$ là RUL tại cycle t. Ví dụ, một động cơ hỏng ở cycle 200 thì tại cycle 170 có RUL bằng 30. Công thức (1) không cắt ngưỡng RUL, nên nhãn ở đầu vòng đời có thể lên tới hàng trăm cycles.

Tập test khác ở chỗ chuỗi bị cắt tại một thời điểm trước khi hỏng. NASA cung cấp riêng RUL thật tại cycle cuối được quan sát của mỗi động cơ test. Vì vậy, đánh giá cuối gồm đúng một dự đoán cho mỗi động cơ test, tại cycle cuối của nó.

1.2 Mục tiêu và câu hỏi nghiên cứu

Mục tiêu của báo cáo là xây dựng một pipeline tái lập được, đi từ dữ liệu thô đến dự đoán RUL, và đánh giá pipeline đó mà không để thông tin của tập test ảnh hưởng tới việc chọn mô hình. Báo cáo trả lời sáu câu hỏi, gồm kiểm tra dữ liệu, xu hướng sensor, parity ETL, so sánh biểu diễn đặc trưng, cảnh báo và chuyển miền:

- **Q1.** Dữ liệu FD001 có đáp ứng các điều kiện hợp lệ về schema, khóa và tính liên tục của cycle hay không?
- **Q2.** Giá trị sensor thay đổi như thế nào giữa đầu và cuối vòng đời động cơ?
- **Q3.** ETL bằng Pandas và PySpark có cho kết quả giống nhau trên cùng input hay không?
- **Q4.** Giữa HGB126 (126 đặc trưng rolling) và HGB169 (126 đặc trưng rolling, `cycle` và 42 đặc trưng lịch sử), biểu diễn nào ổn định hơn trên cùng các dòng OOF và các mốc trong vòng đời FD001?
- **Q5.** Dự đoán RUL có thể chuyển thành cảnh báo sớm cho từng động cơ với chất lượng ra sao?
- **Q6.** Mô hình huấn luyện trên FD001 hoạt động thế nào khi áp dụng lên FD002–FD004?

1.3 Phạm vi và cấu trúc báo cáo

C-MAPSS là dữ liệu mô phỏng, nên mọi kết luận chỉ áp dụng trong môi trường mô phỏng này. Dữ liệu cũng không có chi phí bảo trì, thời gian ngừng máy hay cách quy đổi cycle sang giờ; do

đó báo cáo dừng ở mức dự đoán RUL và mô tả cảnh báo, không đề xuất lịch bảo trì.

Khai báo hỗ trợ công cụ. Trong quá trình thực hiện báo cáo, công cụ AI/LLM được sử dụng để gợi ý cấu trúc, hỗ trợ viết và rà soát một số đoạn mã, notebook, kịch bản kiểm tra và văn bản. Công cụ không được dùng để bịa số liệu, thay thế việc chạy thực nghiệm hoặc tự quyết định kết luận. Người thực hiện trực tiếp kiểm tra mã nguồn, dữ liệu, artifact, trích dẫn và nội dung cuối cùng, đồng thời chịu trách nhiệm về toàn bộ kết quả được trình bày.

Phần còn lại được tổ chức như sau. Mục 2 mô tả dữ liệu và kiểm tra chất lượng. Mục 3 trình bày cơ sở lý thuyết. Mục 4 mô tả cách gom, lưu trữ, truy vấn và biến đổi dữ liệu thành đặc trưng. Mục 5 mô tả mô hình và các protocol đánh giá. Mục 6 trình bày kết quả, Mục 7 thảo luận và nêu giới hạn, Mục 8 kết luận.

2 Dữ liệu

2.1 Nguồn dữ liệu

NASA C-MAPSS mô phỏng các chuỗi vận hành đến khi hỏng của một đội động cơ turbofan cùng loại [8, 11]. Mỗi động cơ bắt đầu với mức hao mòn ban đầu khác nhau, vận hành bình thường, rồi phát sinh lỗi và suy giảm dần đến khi hỏng. Bộ dữ liệu gồm bốn subset khác nhau về số điều kiện vận hành và số dạng lỗi (Bảng 2).

Bảng 2: Quy mô và đặc điểm của bốn subset C-MAPSS.

Subset	Dòng train	Dòng test	Động cơ train	Động cơ test	Điều kiện	Dạng lỗi
FD001	20.631	13.096	100	100	1	1 (HPC)
FD002	53.759	33.991	260	259	6	1 (HPC)
FD003	24.720	16.596	100	100	1	2 (HPC, fan)
FD004	61.249	41.214	249[a]	248[a]	6	2 (HPC, fan)

HPC: High-Pressure Compressor (máy nén áp suất cao). [a]Số động cơ đếm từ file dữ liệu; file `readme.txt` của NASA ghi ngược thành 248 train và 249 test.

FD001 được chọn làm subset chính vì chỉ có một điều kiện vận hành và một dạng lỗi. Khi các yếu tố này cố định, biến động của sensor chủ yếu phản ánh quá trình suy giảm, giúp việc diễn giải từng bước phân tích rõ ràng hơn. FD002–FD004 được dùng trong thí nghiệm bổ sung để kiểm tra mô hình FD001 khi điều kiện vận hành và dạng lỗi thay đổi.

2.2 Cấu trúc và ý nghĩa các cột

Mỗi dòng trong file train và test là một cycle của một động cơ, gồm 26 giá trị cách nhau bởi khoảng trắng: mã động cơ, số thứ tự cycle, ba operational settings và 21 sensor. File gốc không có header; tên cột trong Bảng 3 do repository đặt. NASA chỉ gọi các cột sensor là "sensor measurement" mà không cho biết tên đại lượng vật lý hay đơn vị, vì vậy báo cáo giữ tên `sensor_1` đến `sensor_21` và không suy đoán sensor nào đo nhiệt độ hay áp suất.

Bảng 3: Từ điển dữ liệu của FD001.

Cột	Ý nghĩa	Kiểu và miền giá trị	Cách sử dụng
`split`	Cột thêm khi đọc file, cho biết dòng thuộc train hay test.	`train` hoặc `test`.	Thành phần của khóa; giữ tách biệt hai tập.
`engine_id`	Mã động cơ trong từng tập.	Số nguyên 1–100 ở cả train và test.	Đơn vị chia fold, ghép cặp và tính metric.
`cycle`	Thứ tự cycle của động cơ.	Số nguyên, bắt đầu từ 1, liên tục.	Trục thời gian; dùng để tạo nhãn và window.
`setting_1`–`setting_3`	Ba operational settings.	`float64`; `setting_3` hằng bằng 100 trên train.	Không đưa vào mô hình RUL hiện tại.
`sensor_1`–`sensor_21`	Giá trị đo của 21 sensor trong cycle.	`float64`, không thiếu giá trị.	Nguồn tín hiệu cho EDA, phân tích ghép cặp và đặc trưng.
`rul`	Nhãn RUL train tính theo Công thức (1).	Số nguyên 0–361.	Biến mục tiêu; không có trong file gốc.
`RUL_FD001.txt`	RUL thật tại cycle cuối của mỗi động cơ test.	100 số nguyên; dòng i ứng với động cơ test i.	Chỉ đọc ở bước đánh giá cuối.

Ví dụ, dòng đầu tiên của `train_FD001.txt` có `engine_id=1`, `cycle=1`, ba settings lần lượt là $-0{,}0007$, $-0{,}0004$ và $100{,}0$, tiếp theo là 21 giá trị sensor. Động cơ này hỏng ở cycle 192, nên dòng đầu có `rul=191`.

Hai động cơ có cùng `engine_id` ở train và test là hai động cơ khác nhau. Vì vậy, khóa của một dòng là bộ ba `(split, engine_id, cycle)`, không phải cặp `(engine_id, cycle)`.

2.3 Đặc điểm của tập train FD001

Bảng 4 tóm tắt tập train sau khi gắn nhãn RUL. Không có giá trị thiếu và không có khóa trùng. Điểm đáng chú ý là sáu sensor hằng số trên toàn bộ train, còn `sensor_6` chỉ có hai giá trị và không đổi trong 38/100 động cơ. Chi tiết từng sensor nằm ở Bảng 5.

Bảng 4: Tóm tắt các nhóm cột trên train FD001.

Nhóm cột	Số cột	Thiếu	Nhận xét
`engine_id`, `cycle`	2	0	20.631 dòng của 100 động cơ; khóa không trùng, cycle liên tục.
Operational settings	3	0	`setting_1` có 158 giá trị, `setting_2` có 13 giá trị, `setting_3` hằng bằng 100.
Sensor	21	0	Sáu sensor hằng; `sensor_6` hằng trong 38 động cơ.
Nhãn `rul`	1	0	Từ 0 đến 361 cycles; trung vị theo dòng là 103 cycles.

Bảng 5: Số giá trị khác nhau và miền giá trị của 21 sensor trên train FD001.

Sensor	Số giá trị	Min–max	Ghi chú
sensor_1	1	518,67–518,67	Hằng.
sensor_2	310	641,21–644,53	
sensor_3	3.012	1.571,04–1.616,91	
sensor_4	4.051	1.382,25–1.441,49	Tăng rõ ở cuối vòng đời.
sensor_5	1	14,62–14,62	Hằng.
sensor_6	2	21,60–21,61	Hằng trong 38/100 động cơ.
sensor_7	513	549,85–556,06	
sensor_8	53	2.387,90–2.388,56	Biến thiên hẹp.
sensor_9	6.403	9.021,73–9.244,59	Xu hướng khác nhau giữa các động cơ.
sensor_10	1	1,30–1,30	Hằng.
sensor_11	159	46,85–48,53	Tương quan Spearman âm mạnh với RUL.
sensor_12	427	518,69–523,38	Tương quan Spearman dương mạnh với RUL.
sensor_13	56	2.387,88–2.388,56	Biến thiên hẹp.
sensor_14	6.078	8.099,94–8.293,72	Xu hướng khác nhau giữa các động cơ.
sensor_15	1.918	8,3249–8,5848	
sensor_16	1	0,03–0,03	Hằng.
sensor_17	13	388–400	Giá trị rời rạc.
sensor_18	1	2.388–2.388	Hằng.
sensor_19	1	100–100	Hằng.
sensor_20	120	38,14–39,43	
sensor_21	4.745	22,8942–23,6184	

Sensor hằng không mang thông tin về suy giảm và không có hệ số tương quan xác định. Báo cáo vẫn giữ đủ 21 sensor trong các bảng thống kê để kết quả phản ánh toàn bộ dữ liệu, thay vì chỉ những sensor cho kết quả đẹp.

2.4 Kiểm tra chất lượng và cách xử lý

Trước mọi phân tích, bộ đọc dữ liệu kiểm tra: mỗi dòng có đúng 26 trường, mọi giá trị là số hữu hạn, khóa không trùng, cycle bắt đầu từ 1 và liên tục trong từng động cơ. Với nhãn train, bộ kiểm tra đối chiếu lại Công thức (1). Khi phát hiện lỗi, chương trình dừng với `ContractError` thay vì tự điền hay xóa dòng. Các kiểm tra này quan trọng vì dữ liệu sai vẫn có thể cho ra thống kê trông hợp lý: chẳng hạn chuỗi cycle $1, 2, 4$ không được coi là ba bước liên tiếp khi tạo window. Mỗi lần chạy lưu mã băm SHA-256 của file đầu vào để nhận biết phiên bản dữ liệu đã dùng.

Bảng 6 tóm tắt các đặc điểm dữ liệu cần lưu ý và cách báo cáo xử lý.

Bảng 6: Đặc điểm dữ liệu và cách xử lý.

Đặc điểm	Ảnh hưởng	Cách xử lý
Không thiếu giá trị, khóa hợp lệ	Không cần điền dữ liệu.	Giữ nguyên dữ liệu; dừng nếu kiểm tra thất bại.
Độ dài chuỗi khác nhau (128–362 cycles)	Động cơ sống lâu đóng góp nhiều window hơn.	Chia fold, gán trọng số và tính metric theo động cơ.
Sensor hằng hoặc ít biến thiên	Tương quan và kiểm định không xác định.	Giữ trong bảng, đánh dấu "không xác định".
Ngoại lệ hoặc nhiễu cảm biến	Chưa có ground truth để phân biệt tín hiệu bất thường với suy giảm thật.	Rolling statistics và Health Index chỉ mô tả/làm dịu dao động; không tự động xoá hoặc thay thế giá trị. Đánh giá bộ lọc riêng là hướng tiếp theo.
Không có đơn vị vật lý	Không diễn giải được ý nghĩa vật lý của sensor.	Dùng tên ẩn danh; chênh lệch hiểu theo đơn vị gốc.
Nhãn test tách riêng	Đọc sớm sẽ gây data leakage.	Chỉ đọc `RUL_FD001.txt` sau khi đã ghi dự đoán test.

3 Cơ sở lý thuyết

3.1 Dự đoán RUL và các nghiên cứu liên quan

Prognostics là lĩnh vực nghiên cứu diễn biến suy giảm của hệ thống và thời gian còn lại trước khi hỏng. Saxena và cộng sự xây dựng C-MAPSS để có dữ liệu run-to-failure với nhãn đã biết cho bài toán này [11]. Các nghiên cứu sau đó tiếp cận theo hai hướng chính: trích xuất đặc trưng thủ công rồi dùng mô hình hồi quy, hoặc học trực tiếp từ chuỗi bằng deep learning như CNN và LSTM [6, 7, 22]. Ramasso và Saxena lưu ý rằng kết quả trên C-MAPSS phụ thuộc mạnh vào subset và cách đánh giá [9]; vì vậy, các con số giữa những bài báo dùng cách tiền xử lý hoặc chia dữ liệu khác nhau không so sánh trực tiếp được.

Báo cáo chọn hướng thứ nhất. Đặc trưng thống kê trên window kết hợp với mô hình cây cho phép kiểm tra từng bước của pipeline và phù hợp với trọng tâm của môn học là xử lý dữ liệu tái lập được. Bảng 7 tóm tắt vị trí của báo cáo so với các nguồn tham khảo chính.

Bảng 7: Vị trí của báo cáo so với một số nghiên cứu liên quan.

Nguồn	Nội dung	Liên hệ với báo cáo
Saxena và cộng sự [11]	Mô phỏng suy giảm, tạo C-MAPSS	Cơ sở để hiểu dữ liệu và cách tạo nhãn RUL.
Ramasso và Saxena [9]	Tổng quan các phương pháp trên C-MAPSS	Lý do báo cáo mô tả rõ subset, cách chia dữ liệu và metric.
Lei và cộng sự [6]	Tổng quan machinery prognostics	Bối cảnh chung từ thu thập dữ liệu đến dự đoán RUL.
Li và cộng sự [7]	CNN trên window thời gian	Tham khảo độ dài window 30 cycles và NASA score.
Zheng và cộng sự [22]	LSTM cho dự đoán RUL	Hướng deep learning chưa được thực hiện trong báo cáo.
Saxena và cộng sự [10]	Metric đánh giá prognostics	Phân biệt đánh giá tại một điểm cắt với đánh giá trên toàn chuỗi.

3.2 Metric đánh giá

Với n động cơ, sai số $e_i = \hat{y}_i - y_i$ là RUL dự đoán trừ RUL thật. MAE bằng $\frac{1}{n}\sum |e_i|$ và RMSE bằng $\sqrt{\frac{1}{n}\sum e_i^2}$; cả hai có đơn vị cycle, trong đó RMSE nhạy hơn với các sai số lớn. NASA score cộng điểm phạt của từng dự đoán [7]:

$$S = \sum_{i=1}^{n} s(e_i), \qquad s(e) = \begin{cases} \exp(-e/13) - 1, & e < 0, \\ \exp(e/10) - 1, & e \geq 0. \end{cases} \tag{2}$$

Mẫu số 10 nhỏ hơn 13, nên dự đoán cao hơn RUL thật (báo hỏng muộn) bị phạt nặng hơn dự đoán thấp hơn cùng độ lớn, phản ánh việc báo muộn có thể khiến động cơ hỏng trước khi kịp bảo trì [11]. NASA score không có đơn vị và tăng theo hàm mũ, nên chỉ dùng để so sánh các mô hình trên cùng tập đánh giá.

Theo Saxena và cộng sự [10], một dự đoán tại cycle cuối của mỗi động cơ test chỉ đo chất lượng tại điểm cắt đó, không đo sai số dọc theo toàn bộ vòng đời.

3.3 Trailing window và đặc trưng thống kê

Tại cycle t, mẫu đầu vào của động cơ i là ma trận gồm w cycles gần nhất:

$$W_{i,t} = [\mathbf{x}_{i,t-w+1}, \ldots, \mathbf{x}_{i,t}]^{\mathsf{T}} \in \mathbb{R}^{w \times J}, \qquad t \geq w, \tag{3}$$

trong đó $\mathbf{x}_{i,t}$ là vector $J = 21$ giá trị sensor tại cycle t. Báo cáo dùng $w = 30$, một độ dài từng được sử dụng cho FD001 [7]. Window chỉ chứa quá khứ và hiện tại, nên không có thông tin từ cycle tương lai; ví dụ, mẫu tại cycle 170 dùng cycles 141–170.

Mỗi cột sensor trong window được tóm tắt bằng sáu thống kê: giá trị cuối, trung bình, độ lệch chuẩn mẫu, min, max và độ dốc tuyến tính. Với dãy $z_1, \ldots, z_w$, độ dốc là hệ số của hồi quy bình phương tối thiểu theo vị trí k [5]:

$$b = \frac{\sum_{k=1}^{w}(k - \bar{k})(z_k - \bar{z})}{\sum_{k=1}^{w}(k - \bar{k})^2}, \tag{4}$$

với $\bar{k}$ và $\bar{z}$ là trung bình của vị trí và của giá trị sensor. Đơn vị của b là đơn vị sensor trên một cycle. Giá trị cuối và trung bình cho biết mức tín hiệu, còn độ dốc cho biết tín hiệu đang tăng hay giảm trong 30 cycles gần nhất.

3.4 Kiểm định ghép cặp và hiệu chỉnh đa kiểm định

Khi so sánh một sensor ở đầu và cuối vòng đời của cùng một động cơ, phép ghép cặp loại bỏ khác biệt nền giữa các động cơ. Kiểm định dấu-hạng Wilcoxon xếp hạng độ lớn các chênh lệch rồi so sánh tổng hạng của chênh lệch dương và âm; dưới giả định phân phối chênh lệch đối xứng, nó kiểm định trung vị chênh lệch bằng 0 [21, 18]. Bootstrap lấy mẫu lại có hoàn lại các động cơ để ước lượng khoảng tin cậy cho trung bình chênh lệch mà không cần giả định phân phối chuẩn [4, 16].

Khi kiểm định đồng thời 21 sensor, xác suất có ít nhất một kết quả dương tính giả tăng lên. Thủ tục Benjamini–Hochberg điều chỉnh các p-value để kiểm soát tỷ lệ phát hiện sai kỳ vọng (FDR) trong nhóm kiểm định [3, 17].

3.5 Health Index và ARIMA

Health Index là một chỉ số duy nhất tóm tắt trạng thái của động cơ từ nhiều sensor. Báo cáo tạo Health Index bằng cách chuẩn hóa 21 sensor rồi lấy thành phần chính thứ nhất (PC1) của PCA [13]. Dấu của PC1 được chọn sao cho chỉ số giảm dần theo cycle, tức giá trị thấp ứng với trạng thái xấu hơn.

Chuỗi Health Index h_t của từng động cơ được mô hình hóa bằng ARIMA(p, d, q) [5]:

$$\phi(B)(1 - B)^d h_t = c + \theta(B)\varepsilon_t, \tag{5}$$

trong đó B là toán tử trễ, $\phi(B)$ và $\theta(B)$ là đa thức tự hồi quy bậc p và trung bình trượt bậc q, d là bậc sai phân và ε_t là nhiễu trắng. Kiểm định ADF đánh giá tính dừng của chuỗi [19], còn kiểm định Ljung–Box kiểm tra phần dư sau khi fit còn tự tương quan hay không.

3.6 Spark, Bronze–Silver–Gold và Parquet

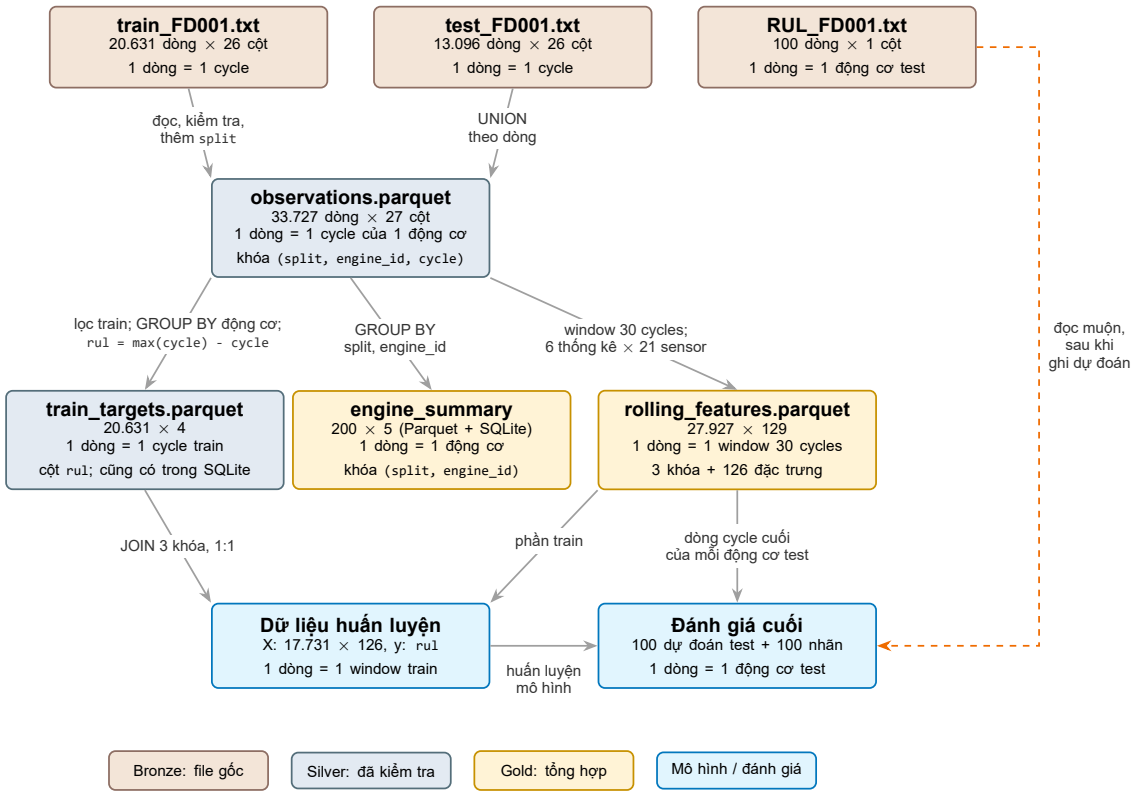
Pipeline dùng PySpark ở chế độ `local[4]` để thực hiện các phép biến đổi trên dữ liệu có schema. DataFrame API hỗ trợ `unionByName` để gom các file quan sát, `groupBy` và `agg` để tạo bảng tóm tắt theo động cơ, `join` để tạo nhãn RUL, cùng các phép sắp xếp theo `(split, engine_id, cycle)` [2]. Các thao tác này được thực thi lại trên cùng input với pipeline Pandas; kết quả được thu về để kiểm tra schema, khóa, số dòng và giá trị trong phép đối chiếu parity. Phần modeling vẫn dùng Pandas và scikit-learn, nên phạm vi Spark được giữ ở ETL và feature pipeline.

Tổ chức Bronze–Silver–Gold tách ba vai trò dữ liệu. Bronze giữ file text gốc và hash nguồn; Silver giữ observations đã kiểm tra kiểu dữ liệu cùng train targets; Gold giữ `engine_summary`, rolling features và các bảng phục vụ truy vấn. Parquet là định dạng lưu trữ theo cột, có encoding và compression cho dữ liệu phân tích [1]; SQLite giữ các bảng nhỏ có index để truy vấn nhanh. Cách phân lớp này giúp mỗi bước có input, output và kiểm tra riêng, đồng thời cho phép thay implementation ETL mà vẫn giữ contract dữ liệu.

4 Tổ chức, truy vấn và biến đổi dữ liệu

Mục này mô tả cách dữ liệu đi từ các file text gốc đến bảng đầu vào của mô hình, theo các câu hỏi về cách gom nhiều file, lưu trữ, truy vấn và biến đổi dữ liệu, qua những bước nào, dùng những cột nào, và các bảng quan hệ với nhau ra sao. Hình 1 tóm tắt toàn bộ luồng; mỗi hộp là một bảng, ghi rõ một dòng biểu diễn điều gì và kích thước thực tế với FD001. Mọi ví dụ trong mục này có thể chạy lại từng bước trong notebook `notebooks/00_data_pipeline_walkthrough.ipynb`; notebook gọi đúng các hàm của pipeline và in kết quả trung gian.

Theo vòng đời quản lý dữ liệu của môn học, pipeline được ánh xạ trực tiếp thành năm bước: **thu thập** các file C-MAPSS và lưu hash nguồn; **làm sạch** bằng schema contract, kiểm tra khóa, cycle và giá trị hữu hạn; **chuẩn hóa** kiểu dữ liệu, tên cột và thứ tự khóa; **tích hợp** train/test bằng UNION, ghép nhãn theo khóa ba thành phần và tạo các bảng Silver–Gold; cuối cùng **lưu trữ** thành Parquet và SQLite để truy vấn bằng Pandas, SQL hoặc PySpark. Như vậy, mỗi bước đều có đầu vào, đầu ra và kiểm tra tương ứng thay vì chỉ mô tả bước tạo đặc trưng.



Hình 1: Luồng dữ liệu từ ba file gốc đến dữ liệu huấn luyện và đánh giá cuối. Chữ trên mũi tên là phép biến đổi; nhãn test chỉ được đọc sau khi dự đoán đã được ghi.

4.1 Gom dữ liệu từ nhiều file

Mỗi subset có ba file. Hai file quan sát `train_FD001.txt` và `test_FD001.txt` có cùng 26 cột nhưng chứa hai nhóm động cơ khác nhau, nên chúng được **nối theo chiều dòng (union)**, không phải join. Trước khi nối, mỗi dòng được gắn thêm cột `split` để biết nó thuộc tập nào (Mã

nguồn 1). Bảng kết quả của FD001 có $20.631 + 13.096 = 33.727$ dòng và 27 cột.

Mã nguồn 1: Đọc và gom hai file quan sát (rút gọn từ `pandas_pipeline.py` và `spark_pipeline.py`).

```
# Pandas: read each file, validate 26 fields, add the split column, then union rows
frames = [read_observations(raw_dir / f"{split}_FD001.txt", split=split)
          for split in ("train", "test")]
observations = pd.concat(frames, ignore_index=True)
observations = observations.sort_values(["split", "engine_id", "cycle"])

# PySpark: the same union on Spark DataFrames
observations = train.unionByName(test).orderBy("split", "engine_id", "cycle")
```

Cột `split` là cần thiết vì train engine 1 và test engine 1 là hai động cơ khác nhau nhưng có cùng `(engine_id, cycle)`. Sau khi nối, cặp `(engine_id, cycle)` bị trùng, còn bộ ba **`(split, engine_id, cycle)`** là duy nhất; đây là khóa của mọi bảng ở cấp cycle. File nhãn `RUL_FD001.txt` không được gom vào bảng này mà được giữ riêng đến bước đánh giá cuối. Bốn subset FD001–FD004 được xử lý ở bốn thư mục riêng; nếu muốn gộp thành một bảng thì phải thêm cột mã subset vào khóa.

4.2 Lưu trữ theo lớp

Bảng 8 liệt kê các bảng được lưu. Parquet lưu dữ liệu theo cột và giữ nguyên kiểu dữ liệu, phù hợp khi phân tích chỉ đọc một vài sensor. SQLite chứa thêm hai bảng nhỏ (`engine_summary`, `train_targets`) có index theo khóa, để truy vấn nhanh bằng SQL.

Bảng 8: Các bảng được lưu cho FD001.

Lớp	File	Một dòng là	Kích thước	Khóa
Bronze	`train_FD001.txt`, `test_FD001.txt`	1 cycle của 1 động cơ	33.727×26	không có header
Silver	`observations.parquet`	1 cycle của 1 động cơ	33.727×27	split, engine_id, cycle
Silver	`train_targets.parquet`	1 cycle train	20.631×4	split, engine_id, cycle
Gold	`engine_summary.parquet`	1 động cơ	200×5	split, engine_id
Gold	`etl.sqlite`	hai bảng trên	–	có index theo khóa
Gold	`rolling_features.parquet`	1 window 30 cycles	27.927×129	split, engine_id, cycle

Bảo vệ dữ liệu và ghi kết quả. Writer xác định thư mục raw thực tế, kể cả đường dẫn tùy chỉnh, để bảo vệ Bronze khi đổi thư mục làm việc. ETL và rolling features được ghi vào staging (thư mục tạm) cùng filesystem trước khi công bố; metadata được công bố sau cùng. Nếu ghi nhãn train thất bại, dữ liệu quan sát và metadata cũ được giữ cùng phiên bản. Lỗi khi công bố được xử lý bằng backup và rollback (khôi phục); nếu rollback cũng lỗi, backup được giữ để

phục hồi thủ công. ETL CLI đưa benchmark và parity được yêu cầu vào cùng bộ kết quả, nên receipt cũ không được coi là bằng chứng cho lần chạy mới.

Kiểm chứng triển khai. Đợt kiểm tra ngày 07/10/2026 trên Windows đạt 426 kiểm thử, bỏ qua một phần kiểm thử TCN do thiếu `torch`[1]. CLI chạy từ thư mục khác vẫn khớp số dòng ở Bảng 8 và giữ nguyên hash Bronze. Môi trường chỉ cài dependency runtime chạy được benchmark Pandas; bốn file requirements khớp `uv.lock`. Hai runner XGBoost và LightGBM dùng chung phần điều phối, lưu nguồn và kiểm tra kết quả; cấu hình mô hình và giao thức đánh giá được giữ nguyên.

4.3 Truy vấn dữ liệu

Dữ liệu được lấy ra theo ba cách, tùy loại bảng.

SQL trên SQLite. Các bảng tổng hợp được truy vấn trực tiếp bằng SQL. Truy vấn trong Mã nguồn 2 join bảng tổng hợp với bảng nhãn để lấy số cycle và RUL của train engine 1 tại ba thời điểm.

Mã nguồn 2: Truy vấn SQLite `outputs/etl/gold/etl.sqlite`.

```
SELECT s.engine_id, s.n_cycles, t.cycle, t.rul
FROM engine_summary AS s
JOIN train_targets  AS t
  ON t.split = s.split AND t.engine_id = s.engine_id
WHERE s.split = 'train' AND s.engine_id = 1 AND t.cycle IN (1, 100, 192);
-- engine_id | n_cycles | cycle | rul
--     1     |   192    |   1   | 191
--     1     |   192    |  100  |  92
--     1     |   192    |  192  |   0
```

Lọc và chọn cột trên Parquet bằng Pandas. Dữ liệu sensor chi tiết được đọc từ Parquet, rồi lọc dòng (selection) và chọn cột (projection). Ví dụ, lấy `sensor_11` của train engine 1 trong cycles 71–100 trả về 30 dòng với giá trị trung bình 47,395.

Spark SQL trên Parquet. Cùng file Parquet có thể được đăng ký thành bảng tạm và truy vấn bằng Spark SQL, gồm cả `GROUP BY`, `JOIN` và window function. Mã nguồn 3 dùng window function để tính trung bình 30 cycles gần nhất của `sensor_11`, chính là phép tính của đặc trưng `sensor_11__mean`. Kết quả tại cycle 100 (47,395) trùng với kết quả Pandas. Phép lọc cycle phải nằm ở truy vấn ngoài: nếu đặt cùng cấp với window function, window chỉ nhìn thấy các dòng đã lọc và cho kết quả sai. Spark chạy ở chế độ local trên một máy; đây là minh họa cách truy vấn, không phải phép đo hiệu năng phân tán.

[1] Bằng chứng kiểm tra và hash mã nguồn: `handoff/CODE_RELIABILITY_20261007.json`.

Mã nguồn 3: Spark SQL trên `observations.parquet`: trung bình 30 cycles gần nhất.

```sql
SELECT engine_id, cycle, n_rows_in_window, ROUND(mean30, 3) AS mean30
FROM (
  SELECT engine_id, cycle,
         COUNT(*)       OVER w AS n_rows_in_window,
         AVG(sensor_11) OVER w AS mean30
  FROM observations
  WHERE split = 'train' AND engine_id = 1
  WINDOW w AS (PARTITION BY split, engine_id ORDER BY cycle
               ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)
)
WHERE cycle IN (29, 30, 100);
-- cycle 29:  29 rows, mean30 = 47.279   (window not full: no feature row)
-- cycle 30:  30 rows, mean30 = 47.283
-- cycle 100: 30 rows, mean30 = 47.395
```

4.4 Các bước biến đổi

Biến đổi ở đây là bước T trong ETL: đổi kiểu, gom dòng, tổng hợp, tạo nhãn và tạo đặc trưng. Bảng 9 liệt kê sáu bước theo thứ tự, kèm kích thước bảng sau mỗi bước.

Bảng 9: Sáu bước biến đổi dữ liệu và kích thước sau mỗi bước (FD001).

#	Đầu vào → đầu ra	Phép biến đổi	Kết quả
1	2 file text → 2 bảng có kiểu	Tách 26 trường, gán tên cột và kiểu; kiểm tra khóa và cycle liên tục; thêm `split`	20.631 và 13.096 dòng
2	2 bảng → observations	UNION theo dòng, sắp theo khóa	33.727×27
3	observations train → train_targets	GROUP BY động cơ, lấy max(cycle); rul = max(cycle) − cycle	20.631×4
4	observations → engine_summary	GROUP BY `(split, engine_id)`: đếm cycle, lấy cycle đầu và cuối	200×5
5	observations → rolling_features	Với mỗi động cơ: window 30 cycles, 6 thống kê × 21 sensor	27.927×129
6	đặc trưng train + nhãn → X, y	JOIN theo đủ 3 khóa, bắt buộc một-một	X: 17.731×126

Mã nguồn 4 trích phần code của bước 3, 5 và 6. Bước 3 được viết bằng cả Pandas và PySpark với cùng logic.

Mã nguồn 4: Tạo nhãn, tạo đặc trưng và join (trích từ `targets.py`, `spark_pipeline.py`, `rolling.py`, `modeling/pipeline.py`).

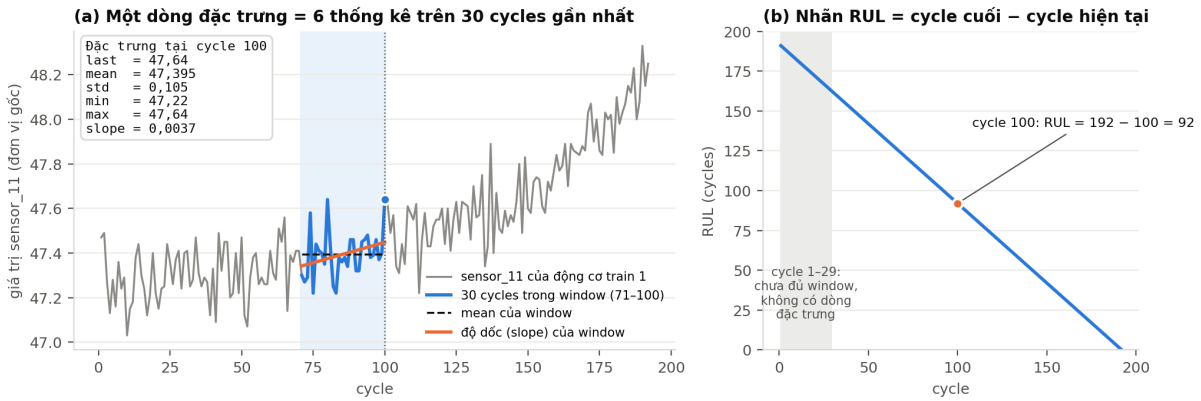
```python
# Step 3 - Pandas: RUL = last cycle of the engine - current cycle
last_cycle = observations.groupby("engine_id")["cycle"].transform("max")
targets["rul"] = last_cycle - observations["cycle"]
```

```
# Step 3 - PySpark: the same label with groupBy + join
max_cycles = train.groupBy("engine_id").agg(F.max("cycle").alias("max_cycle"))
targets = (train.join(max_cycles, on="engine_id")
                .withColumn("rul", F.col("max_cycle") - F.col("cycle")))

# Step 5 - for every (split, engine_id) group, sorted by cycle
rolling = values.rolling(30, min_periods=30)
feature[f"{sensor}__last"]  = values
feature[f"{sensor}__mean"]  = rolling.mean()
feature[f"{sensor}__std"]   = rolling.std(ddof=1)
feature[f"{sensor}__min"]   = rolling.min()
feature[f"{sensor}__max"]   = rolling.max()
feature[f"{sensor}__slope"] = rolling.apply(_slope, raw=True)

# Step 6 - one-to-one join on the full key
dataset = train_features.merge(train_targets, on=["split", "engine_id", "cycle"],
                               how="left", validate="one_to_one")
```

Hình 2 minh họa bước 5 và bước 3 trên train engine 1. Tại cycle 100, window gồm cycles 71–100; sáu thống kê của `sensor_11` là giá trị cuối 47,64, trung bình 47,395, độ lệch chuẩn 0,105, min 47,22, max 47,64 và độ dốc 0,0037 mỗi cycle. Nhãn của dòng này là $192 - 100 = 92$. Giá trị ở cycle 101 trở đi không tham gia tính toán. Vì 29 cycle đầu chưa đủ window, một động cơ dài T cycles tạo ra $T - 29$ dòng đặc trưng; với FD001 là $20.631 - 100 \times 29 = 17.731$ dòng train và $13.096 - 100 \times 29 = 10.196$ dòng test.



Hình 2: Minh họa bước tạo đặc trưng và nhãn trên train engine 1. (a) Sáu thống kê của `sensor_11` trên window cycles 71–100 tạo thành phần đặc trưng của dòng cycle 100. (b) Nhãn RUL giảm tuyến tính về 0 tại cycle hỏng.

ETL chỉ chuẩn hóa schema và kiểu dữ liệu, không scale giá trị sensor. Việc scale chỉ diễn ra bên trong mô hình Ridge (StandardScaler fit trong từng fold) và khi tạo Health Index; HGB dùng đặc trưng ở thang đo gốc.

Để kiểm tra tính đúng của ETL, PySpark thực hiện lại bước 1–4 và bộ đối chiếu so sánh

schema, khóa, số dòng và từng giá trị với kết quả Pandas. Ngoài ra, benchmark ETL lịch sử đo thời gian Pandas thực hiện bước 4 khi dữ liệu được nhân bản 1, 10 và 100 lần trong bộ nhớ. Benchmark này chỉ đo một phép tổng hợp, không đo đọc ghi file và không đo PySpark; dữ liệu nhân bản không được dùng để huấn luyện. Benchmark Pandas–PySpark bổ sung có đọc/ghi và rolling features được trình bày riêng ở Mục 7.7.

4.5 Cột được dùng cho mô hình

Bảng 10 cho biết mỗi nhóm cột được dùng vào việc gì. Mô hình chỉ nhận 126 cột đặc trưng, đặt tên theo mẫu `<sensor>__<thống kê>`, ví dụ `sensor_11__mean`. Ba cột khóa chỉ dùng để join và chia fold theo động cơ. Ba operational settings không nằm trong tập đặc trưng: FD001 chỉ có một điều kiện vận hành nên settings gần như không đổi; với FD002 và FD004, lựa chọn này trở thành một giới hạn (Mục 7.3).

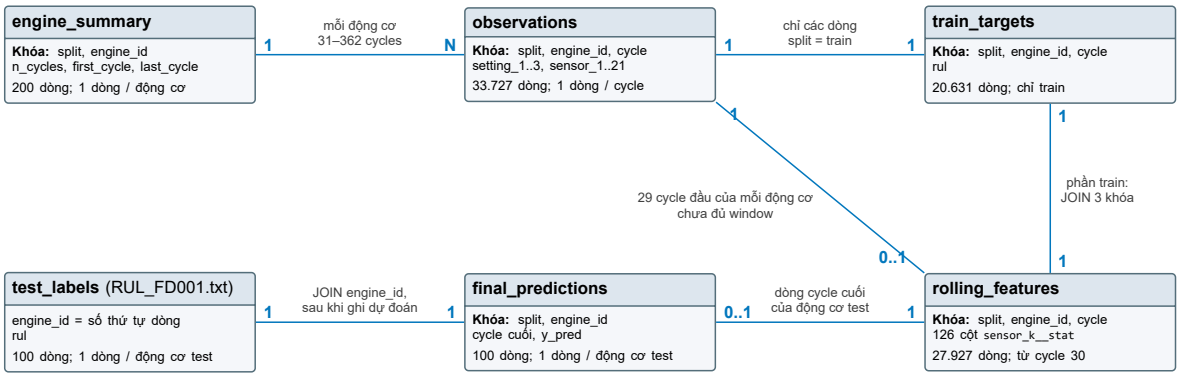
Bảng 10: Vai trò của các nhóm cột.

Nhóm cột	Số cột	Vai trò	Trong X?
`split`, `engine_id`, `cycle`	3	Khóa: join, chia fold theo động cơ	Không
`setting_1`–`setting_3`	3	Điều kiện vận hành	Không
`sensor_1`–`sensor_21`	21	Tín hiệu gốc, chỉ dùng qua đặc trưng window	Không
`sensor_k__{last, mean, std, min, max, slope}`	126	Đặc trưng window 30 cycles	Có
`rul`	1	Nhãn (biến mục tiêu y)	Là y

Báo cáo không thực hiện chọn lọc đặc trưng (feature selection): toàn bộ 126 cột được giữ, kể cả các cột sinh ra từ sensor hằng. Cách làm này tránh dùng kết quả EDA hoặc p-value trên toàn bộ train để chọn cột, vốn là một dạng data leakage đối với cross-validation.

4.6 Quan hệ giữa các bảng

Hình 3 và Bảng 11 mô tả quan hệ giữa các bảng. Quan hệ duy nhất có bậc nhiều là giữa động cơ và quan sát (**1:N**): một động cơ có từ 31 đến 362 cycles. Các quan hệ còn lại là **1:1** hoặc **1:0..1**. Mỗi quan hệ được kiểm tra trực tiếp trên dữ liệu trong notebook walkthrough; kết quả ghi ở cột cuối của bảng.



Hình 3: Quan hệ giữa các bảng trong một subset. Số ở hai đầu mỗi đường là bậc quan hệ; "0..1" nghĩa là một dòng bên kia có thể có hoặc không có dòng tương ứng.

Bảng 11: Quan hệ giữa các bảng và kết quả kiểm tra trên FD001.

Hai bảng	Quan hệ	Kiểm tra trên dữ liệu
engine_summary – observations	1 : N	Mỗi động cơ có 31–362 cycles
observations (train) – train_targets	1 : 1	20.631 / 20.631 dòng khớp khóa
observations – rolling_features	1 : 0..1	27.927 dòng có đặc trưng; 5.800 dòng (29 cycle đầu × 200 động cơ) không có
rolling_features (train) – train_targets	1 : 1	17.731 dòng sau join, đúng bằng số dòng đặc trưng train
final_predictions – test_labels	1 : 1	100 dòng, 100 động cơ test khác nhau

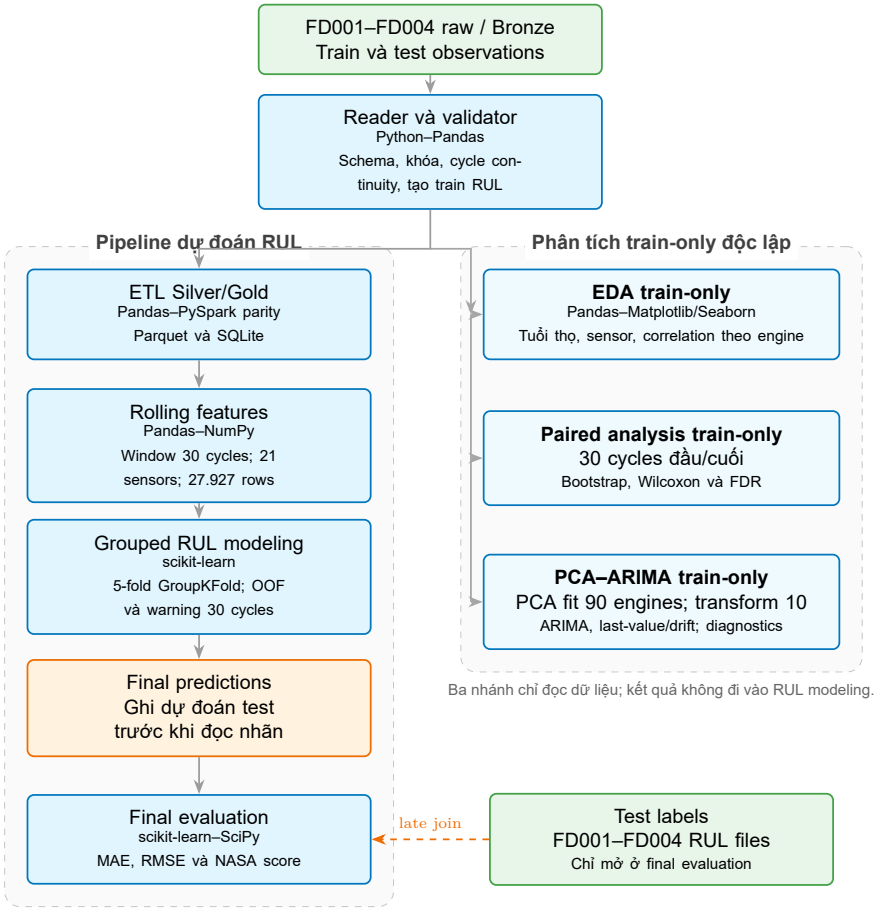
Không có quan hệ nhiều-nhiều. Một window dùng 30 quan sát, và hai window liên tiếp dùng chung 29 quan sát. Đây là phụ thuộc theo thời gian khi tính toán, không phải một phép join nhiều-nhiều giữa hai bảng: mỗi dòng đặc trưng vẫn gắn với đúng một cycle cuối của window.

Việc join đúng khóa quan trọng vì một lỗi nhỏ có thể làm nhân số dòng. Với train engine 1 (163 dòng đặc trưng, 192 dòng nhãn), join chỉ theo `engine_id` ghép mọi dòng đặc trưng với mọi dòng nhãn, tạo ra $163 \times 192 = 31.296$ dòng; join theo đủ ba khóa trả về đúng 163 dòng. SQLite chưa khai báo ràng buộc khóa ngoại (foreign key); thay vào đó, code kiểm tra quan hệ một-một bằng tham số `validate="one_to_one"` và dừng nếu vi phạm.

5 Phương pháp

5.1 Tổng quan pipeline

Hình 4 mô tả toàn bộ luồng xử lý. Sau khi đọc và kiểm tra, dữ liệu đi theo hai nhánh. Nhánh chính (bên trái) gồm ETL, tạo đặc trưng, huấn luyện và đánh giá mô hình RUL. Nhánh phụ (bên phải) gồm EDA, phân tích ghép cặp đầu/cuối vòng đời và Health Index–ARIMA. Nhánh phụ chỉ đọc tập train và không cung cấp đặc trưng cho mô hình RUL, nên kết quả của nó không ảnh hưởng tới đánh giá mô hình. Nhãn test chỉ được ghép ở bước cuối, sau khi dự đoán test đã được ghi ra file.



Hình 4: Luồng xử lý dữ liệu. Nhánh train-only bên phải không đi vào mô hình RUL; nhãn test chỉ được ghép ở bước đánh giá cuối.

Bảng 12 liệt kê công cụ ở từng bước. Pandas là implementation chính; PySpark thực hiện lại phần ETL để đối chiếu kết quả. Cách gom, lưu trữ, truy vấn và biến đổi dữ liệu của nhánh chính đã được trình bày chi tiết ở Mục 4; các mục dưới đây mô tả mô hình và các protocol đánh giá.

Bảng 12: Công cụ và đầu ra của từng bước.

Bước	Công cụ	Đầu ra
Đọc và kiểm tra	Python, Pandas	Bảng quan sát đã kiểm tra, nhãn RUL train
ETL	Pandas, PySpark, Parquet, SQLite	Lớp Silver/Gold, báo cáo parity
Tạo đặc trưng	Pandas, NumPy	126 đặc trưng rolling; alignment audit thêm `cycle` và 42 đặc trưng lịch sử
Mô hình	scikit-learn	GroupKFold, Ridge, HistGradientBoosting
Đánh giá	scikit-learn, SciPy	MAE, RMSE, NASA score, khoảng tin cậy bootstrap
Phân tích chuỗi	statsmodels	ARIMA, ADF, Ljung–Box
Trực quan hóa	Matplotlib, Seaborn, Streamlit	Hình trong báo cáo, dashboard chỉ đọc kết quả

5.2 Huấn luyện và đánh giá mô hình RUL

Ba mô hình được so sánh:

- **Median baseline**: dự đoán một hằng số bằng trung vị có trọng số của RUL trong fold huấn luyện.
- **Ridge**: hồi quy tuyến tính với phạt L2, $\alpha = 1$; đặc trưng được chuẩn hóa bằng `StandardScaler` fit trong từng fold huấn luyện.
- **HistGradientBoosting (HGB126)**: tổ hợp cây hồi quy với 200 vòng boosting, learning rate 0,05, tối đa 31 lá mỗi cây, tối thiểu 20 mẫu mỗi lá, tắt early stopping, `random_state=42`; dùng đặc trưng ở thang đo gốc.

Mô hình được chọn bằng GroupKFold 5 fold theo `engine_id`: mỗi fold có 20 động cơ, và mọi window của một động cơ nằm cùng một fold. Chia ngẫu nhiên theo dòng sẽ đưa các window gần như trùng nhau của cùng động cơ vào cả train và validation, làm sai số validation thấp một cách giả tạo. Vì động cơ sống lâu có nhiều window hơn, trọng số mẫu được chuẩn hóa để tổng trọng số của mỗi động cơ bằng 1; nếu không, vài động cơ dài sẽ chi phối hàm mất mát.

Tiêu chí chọn là MAE trung bình theo động cơ: tính MAE trên các window của từng động cơ, lấy trung bình trên các động cơ trong fold, rồi lấy trung bình qua 5 fold. RMSE và NASA score được ghi kèm để tham khảo. Mô hình có MAE thấp nhất được huấn luyện lại trên toàn bộ 17.731 window train và dự đoán RUL tại cycle cuối của mỗi động cơ test. Chỉ sau khi 100 dự đoán này đã được ghi ra file, pipeline mới đọc `RUL_FD001.txt` để tính metric cuối. Khoảng tin cậy 95% của metric test được ước lượng bằng bootstrap 10.000 lần, lấy mẫu lại động cơ có hoàn lại, seed 42.

Alignment audit giữa hai biểu diễn đặc trưng. Sau protocol lịch sử, báo cáo chấm lại các dự đoán đã lưu của HGB126 và HGB169 trên cùng 17.731 dòng, cùng bộ khóa (`split, engine_id, cycle`), cùng nhãn RUL và cùng các fold theo động cơ. HGB126 dùng 126 đặc trưng rolling của window 30 cycles. HGB169 dùng đúng 126 đặc trưng đó, thêm cột `cycle` và 42 đặc trưng lịch

sử từ 30 cycles đầu, tổng cộng 169 đặc trưng; mọi đặc trưng đều được tính nhân quả. So sánh dùng paired bootstrap 10.000 lần trên `engine_id`, điều kiện trên dự đoán đã lưu, nên khoảng tin cậy không bao gồm bất định do fit lại, random seed hoặc chọn mô hình. Panel trajectory dùng toàn bộ 17.731 dòng; panel endpoint lấy các mốc $0{,}5$, $0{,}7$ và $0{,}9$ tuổi thọ của mỗi động cơ. Kết quả này dùng để chọn predictor candidate cho báo cáo; không đọc nhãn test để thực hiện lựa chọn. Metric test của các run lịch sử chỉ được báo cáo tách riêng.

5.3 Cảnh báo sớm

Để minh họa cách dùng dự đoán RUL, pipeline chuyển dự đoán OOF trên train thành cảnh báo: động cơ được cảnh báo tại cycle đầu tiên mà RUL dự đoán không vượt quá 30 cycles. Ngưỡng 30 được cố định trước, không tối ưu trên dữ liệu. Tập train được dùng vì mỗi động cơ có toàn bộ quỹ đạo đến khi hỏng, trong khi test chỉ có một điểm cắt.

Với động cơ i, đặt a_i là cycle cảnh báo đầu tiên ($\min\{t : \hat{y}_{i,t} \le 30\}$) và r_i là cycle đầu tiên RUL thật chạm ngưỡng ($\min\{t : y_{i,t} \le 30\}$). Hai đại lượng được ghi lại là:

- **Lead time** $\ell_i = T_i - a_i$: số cycle từ cảnh báo đến khi hỏng.
- **Độ lệch so với ngưỡng** $\delta_i = a_i - r_i$: $\delta_i < 0$ là cảnh báo trước ngưỡng, $\delta_i = 0$ là đúng ngưỡng, $\delta_i > 0$ là sau ngưỡng (cảnh báo trễ).

Động cơ không bao giờ có dự đoán dưới ngưỡng được ghi là bỏ lỡ. Vì mọi động cơ train đều đi đến hỏng, không tồn tại động cơ "âm tính", nên tỷ lệ báo động giả ở cấp động cơ không xác định. Ngoài ra, cùng tập OOF này đã được dùng để chọn mô hình, nên kết quả cảnh báo là đánh giá nội bộ, có phần lạc quan.

5.4 Phân tích ghép cặp đầu/cuối vòng đời

Phân tích này trả lời câu hỏi Q2 trên tập train. Với sensor s của động cơ i, báo cáo tính trung bình của 30 cycles đầu và 30 cycles cuối, rồi lấy chênh lệch:

$$E_{i,s} = \frac{1}{30}\sum_{t=1}^{30} x_{i,t,s}, \qquad L_{i,s} = \frac{1}{30}\sum_{t=T_i-29}^{T_i} x_{i,t,s}, \qquad \Delta_{i,s} = L_{i,s} - E_{i,s}. \tag{6}$$

Với động cơ hỏng ở cycle 200, đoạn đầu là cycles 1–30 và đoạn cuối là cycles 171–200. Động cơ ngắn nhất dài 128 cycles, nên hai đoạn không bao giờ chồng nhau. Lấy trung bình 30 cycles thay vì một giá trị đơn lẻ giúp giảm biến động ngắn hạn trong mô tả đầu/cuối; dữ liệu không có ground truth cảm biến sạch để khẳng định đây là khử nhiễu. Mỗi động cơ đóng góp đúng một cặp cho mỗi sensor, tổng cộng $100 \times 21 = 2.100$ cặp.

Đại lượng chính là trung bình chênh lệch $\widehat{\Delta}_s = \frac{1}{100}\sum_i \Delta_{i,s}$, có cùng đơn vị với sensor. Cấu hình suy luận được cố định trước khi chạy:

- Khoảng tin cậy 95% của $\widehat{\Delta}_s$: bootstrap percentile, 10.000 lần lấy mẫu lại động cơ, seed 42.
- Kiểm định Wilcoxon hai phía, dạng permutation với 100.000 hoán vị; chênh lệch bằng 0 bị loại khỏi phép xếp hạng (`zero_method="wilcox"`).

- Hiệu chỉnh Benjamini–Hochberg với $q = 0{,}05$ trên cả nhóm 21 sensor. Sensor có toàn bộ chênh lệch bằng 0 được giữ trong bảng với trạng thái "không kiểm định được".

Vì các sensor có thang đo khác nhau, độ lớn $\widehat{\Delta}_s$ không so sánh được giữa hai sensor. Kết quả mô tả sự thay đổi trên train và không được dùng để chọn đặc trưng cho mô hình.

5.5 Health Index và dự báo ARIMA

Nhánh này khảo sát liệu một chỉ số sức khỏe tổng hợp có dự báo được theo thời gian hay không. Mười động cơ train được chọn trước khi chạy dự báo: hai động cơ trong mỗi fold GroupKFold, tại các vị trí cách đều theo tuổi thọ. StandardScaler, PCA một thành phần và hướng của PC1 được fit trên 90 động cơ còn lại; mười động cơ được chọn chỉ được biến đổi theo các tham số đã fit.

Chuỗi Health Index của mỗi động cơ được chia theo thời gian: 70% đầu để fit ARIMA và 30% cuối để đánh giá dự báo. Bậc mô hình được chọn bằng AIC trên lưới $p, q \in \{0, 1, 2\}$ và $d \in \{0, 1\}$, chỉ dùng đoạn fit. Đầu ra gồm MAE, RMSE, tỷ lệ điểm thật nằm trong khoảng dự đoán 95%, kiểm định ADF trên đoạn fit và Ljung–Box trên phần dư (bậc tự do trừ đi $p + q$). Để có mốc so sánh, hai baseline được tính trên cùng đoạn: `last_value` giữ nguyên giá trị cuối của đoạn fit, còn `drift` kéo dài đường thẳng nối điểm đầu và điểm cuối của đoạn fit.

5.6 Đánh giá chuyển miền sang FD002–FD004

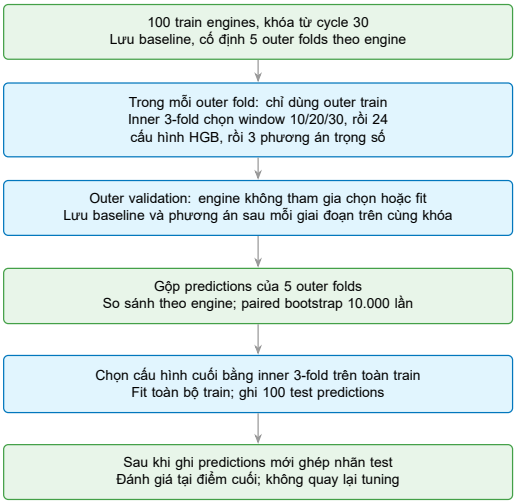
Thí nghiệm này trả lời câu hỏi Q6. Window 30 không dùng được cho mọi subset vì động cơ test ngắn nhất của FD004 chỉ có 19 cycles, nên toàn bộ ma trận dùng window 10 cycles. Mỗi subset được đánh giá theo hai cách:

- **Nội bộ**: chọn mô hình bằng GroupKFold, huấn luyện lại và đánh giá trên train và test của chính subset đó.
- **Chuyển miền**: mô hình được chọn và huấn luyện trên toàn bộ train FD001, giữ cố định rồi dự đoán test của FD002, FD003 và FD004.

Các bước còn lại, gồm đặc trưng, tiêu chí chọn mô hình, cách đọc nhãn test muộn và bootstrap theo động cơ, giống protocol chính. Vì window khác nhau, kết quả của ma trận này không so sánh trực tiếp với MAE 15,3490 cycles của FD001.

5.7 Thử nghiệm cải thiện bằng nested cross-validation

Sau khi có kết quả chính, báo cáo thử ba hướng cải thiện độ chính xác ở vùng RUL thấp, nơi cảnh báo được phát ra: (1) đổi độ dài window, (2) chỉnh siêu tham số của HGB, và (3) tăng trọng số cho mẫu có RUL thấp. Để đánh giá công bằng một quy trình có bước chọn cấu hình, thí nghiệm dùng nested GroupKFold (Hình 5) [15, 12]. Vòng ngoài giữ nguyên 5 fold của mô hình gốc; trong mỗi vòng ngoài, một inner GroupKFold 3 fold chỉ dùng động cơ của phần train để chọn cấu hình, sau đó cấu hình được chọn mới được đánh giá trên các động cơ validation vòng ngoài.



Hình 5: Quy trình nested cross-validation. Mọi bước chọn window, siêu tham số và trọng số đều nằm trong vòng trong.

Việc chọn diễn ra tuần tự theo ba giai đoạn:

1. **Window**: 10, 20 hoặc 30 cycles. Cả ba cùng tính trên 17.731 dòng từ cycle 30 trở đi, nên số mẫu so sánh không đổi. Window 50 không được xét vì 7 động cơ test FD001 có ít hơn 50 cycles.
2. **Siêu tham số**: 24 cấu hình, gồm cấu hình gốc và 23 cấu hình lấy mẫu ngẫu nhiên (seed 42) từ learning rate $\{0{,}03; 0{,}05; 0{,}1\}$, số vòng boosting $\{100; 200; 400\}$, số lá tối đa $\{15; 31; 63\}$, số mẫu tối thiểu mỗi lá $\{10; 20; 40\}$ và hệ số L2 $\{0; 0{,}01; 0{,}1\}$ [14].
3. **Trọng số**: hệ số $(1; 1; 1)$, $(1; 1{,}5; 2)$ hoặc $(1; 2; 3)$ cho các vùng RUL > 30, $15 <$ RUL ≤ 30 và RUL ≤ 15; sau đó chuẩn hóa để tổng trọng số mỗi động cơ vẫn bằng 1.

Tiêu chí được chốt trước khi chạy. Metric chính là **MAE ở vùng RUL thấp**, tức MAE trên các dòng có RUL thật ≤ 30, tính theo động cơ rồi lấy trung bình. Một cấu hình chỉ được chấp nhận nếu đồng thời không làm tăng ba chỉ số kiểm soát so với cấu hình đang dùng: MAE toàn bộ, tỷ lệ động cơ cảnh báo trễ hoặc bỏ lỡ (theo Mục 5.3), và tỷ lệ báo sớm, tức tỷ lệ dòng có RUL dự đoán ≤ 30 trong khi RUL thật > 30. Chênh lệch giữa cấu hình mới và mô hình gốc được ước lượng bằng paired bootstrap trên cùng các động cơ, 10.000 lần. Cuối cùng, cùng quy trình chọn được chạy lại trên toàn bộ train để ra cấu hình cuối, rồi mới dự đoán test.

Cần lưu ý rằng nhãn test FD001 đã được dùng để đánh giá mô hình gốc trước thí nghiệm này. Thí nghiệm không dùng test để chọn cấu hình, nhưng không thể coi test là hoàn toàn "chưa từng thấy".

6 Kết quả

Các mục từ 6.1 đến 6.7 trình bày kết quả trên FD001. Mục 6.9 trình bày ma trận chuyển miền dùng window 10 cycles; các mục cuối trình bày Health Index–ARIMA và ETL.

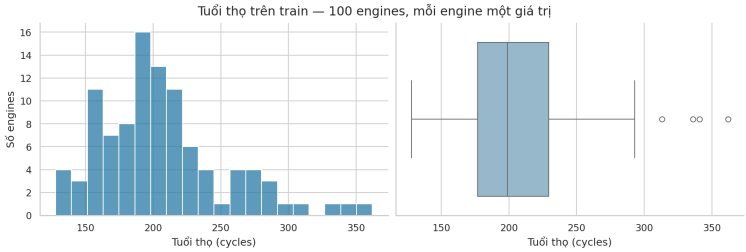
6.1 Kiểm tra dữ liệu và phân tích khám phá

Cả ba bảng (quan sát train, nhãn train, quan sát test) đều vượt qua mọi kiểm tra về schema, khóa, kiểu số và tính liên tục của cycle. Tập train có 20.631 dòng và tập test có 13.096 dòng, mỗi tập 100 động cơ; 100 nhãn test hợp lệ về cấu trúc.

Tuổi thọ của 100 động cơ train dao động từ 128 đến 362 cycles (Bảng 13, Hình 6). Phân phối lệch phải: trung bình 206,31 cycles lớn hơn trung vị 199 cycles do một số động cơ sống rất lâu.

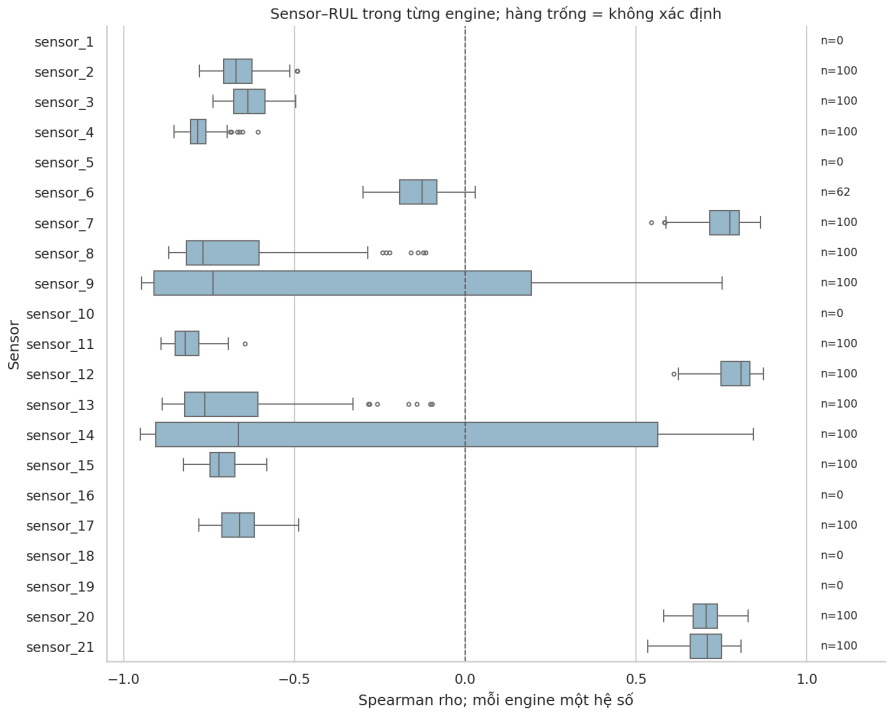
Bảng 13: Thống kê tuổi thọ của 100 động cơ train FD001, đơn vị cycle.

Thống kê	Giá trị	Thống kê	Giá trị
Trung bình	206,31	Độ lệch chuẩn	46,34
Nhỏ nhất	128	Lớn nhất	362
Tứ phân vị 25%	177	Trung vị	199
Tứ phân vị 75%	229,25	Số động cơ	100



Hình 6: Phân phối tuổi thọ của các động cơ train; mỗi động cơ đóng góp một giá trị.

Tương quan Spearman giữa từng sensor và RUL được tính riêng trong mỗi động cơ, rồi tóm tắt qua 100 động cơ (Hình 7). Cách tính này tránh để động cơ dài chi phối kết quả. Sáu sensor hằng (1, 5, 10, 16, 18, 19) không có tương quan xác định. Nhiều sensor có tương quan mạnh và nhất quán giữa các động cơ, ví dụ `sensor_11` (trung vị $-0{,}8190$) và `sensor_12` (trung vị $0{,}8075$), tức `sensor_11` tăng và `sensor_12` giảm khi động cơ tiến gần đến hỏng. Ngược lại, `sensor_9` và `sensor_14` có tương quan âm ở phần lớn động cơ nhưng dương ở một số động cơ khác. `sensor_6` chỉ xác định được ở 62 động cơ, với trung vị yếu $-0{,}1264$.



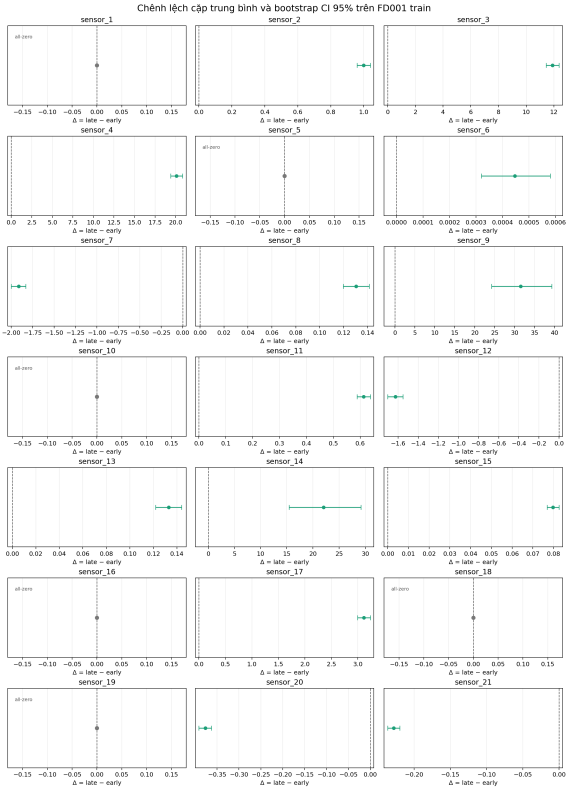
Hình 7: Phân phối hệ số tương quan Spearman sensor–RUL qua các động cơ train.

6.2 Thay đổi của sensor giữa đầu và cuối vòng đời

Trong 21 sensor, sáu sensor hằng có toàn bộ chênh lệch bằng 0 và không kiểm định được. Với 15 sensor còn lại, kiểm định Wilcoxon bác bỏ giả thuyết không sau hiệu chỉnh Benjamini–Hochberg; p-value đã hiệu chỉnh của cả 15 sensor đều xấp xỉ 0,000028. Giá trị giống nhau này là giới hạn độ phân giải của 100.000 hoán vị, không có nghĩa là các sensor thay đổi như nhau. Vì vậy, độ lớn thay đổi cần được đọc từ trung bình chênh lệch và khoảng tin cậy (Bảng 14, Hình 8).

Bảng 14: Một số kết quả tiêu biểu của phân tích ghép cặp; chênh lệch bằng trung bình 30 cycles cuối trừ trung bình 30 cycles đầu, theo đơn vị gốc của sensor.

Sensor	Trung bình chênh lệch	Khoảng tin cậy 95%	Hướng
sensor_4	20,128097	[19,402735; 20,865606]	Tăng ở 100/100 động cơ
sensor_7	−1,913527	[−1,999307; −1,829684]	Giảm ở 100/100 động cơ
sensor_11	0,613257	[0,589017; 0,637944]	Tăng ở 100/100 động cơ
sensor_12	−1,627190	[−1,704288; −1,552119]	Giảm ở 100/100 động cơ
sensor_20	−0,377007	[−0,391534; −0,363173]	Giảm ở 100/100 động cơ



Hình 8: Trung bình chênh lệch đầu/cuối vòng đời và khoảng tin cậy bootstrap 95% của từng sensor.

Các sensor 4, 7, 11, 12 và 20 thay đổi cùng hướng ở tất cả 100 động cơ, cho thấy một xu hướng suy giảm rất nhất quán trong FD001. `sensor_6` là trường hợp ngược lại: dù có ý nghĩa thống kê, 53/100 động cơ có chênh lệch bằng 0 và trung bình chênh lệch chỉ khoảng 0,00045. Đây là ví dụ cho thấy p-value nhỏ không đồng nghĩa với thay đổi lớn.

6.3 Mô hình RUL

Bảng 15 so sánh ba mô hình trong protocol lịch sử trên 5 fold. HGB126 có MAE thấp nhất (20,2143 cycles), thấp hơn Ridge 3,78 cycles và thấp hơn median baseline hơn một nửa. Đây là baseline lịch sử của pipeline 126 đặc trưng; alignment audit ở tiểu mục kế tiếp so sánh nó với HGB169 trên cùng các dòng và cùng fold. Độ lệch chuẩn MAE giữa các fold của HGB126 và Ridge đều dưới 1 cycle, nhỏ hơn khoảng cách giữa hai mô hình.

Bảng 15: Kết quả GroupKFold 5 fold trên 17.731 window train. MAE và RMSE là trung bình giữa 5 fold, tính bằng cycles; độ lệch chuẩn chỉ áp dụng cho MAE.

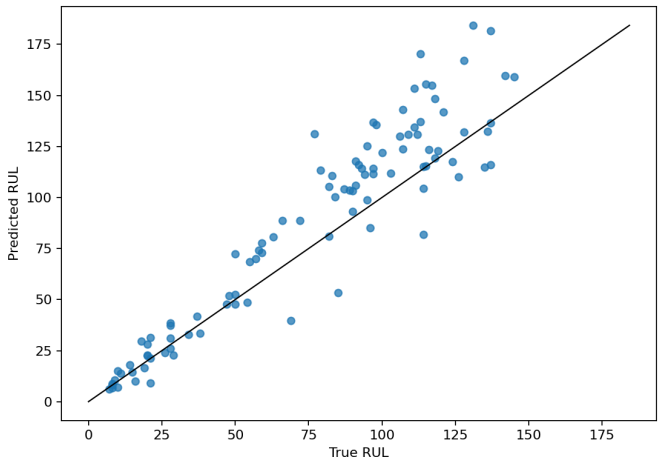
Mô hình	MAE theo động cơ	Độ lệch chuẩn MAE	RMSE (TB fold)	Vai trò
Median baseline	46,9543	0,1193	62,9006	—
Ridge ($\alpha = 1$)	23,9922	0,9479	35,9467	—
HGB126	20,2143	0,9821	33,5479	lịch sử

Sau khi huấn luyện lại trên toàn bộ train, HGB126 đạt MAE 15,3490 cycles trên 100 động cơ test (Bảng 16). Đây là metric test của baseline lịch sử, được giữ riêng với alignment audit vì

các run đã được lưu từ trước. MAE test thấp hơn MAE cross-validation vì hai con số đo trên các tập điểm khác nhau: cross-validation tính trên mọi window từ cycle 30 đến khi hỏng, gồm cả các window đầu đời có RUL lớn và khó dự đoán; test chỉ có một điểm ở cuối mỗi chuỗi bị cắt, với RUL thật từ 7 đến 145 cycles.

Bảng 16: Metric test lịch sử của HGB126 trên 100 động cơ FD001; khoảng tin cậy bootstrap 95% lấy mẫu lại động cơ 10.000 lần.

Metric	Giá trị	Khoảng tin cậy 95%
MAE (cycles)	15,3490	[12,7286; 18,0866]
RMSE (cycles)	20,5413	[17,3442; 23,5850]
NASA score	1.513,2577	[743,4999; 2.476,2743]



Hình 9: RUL thật và RUL dự đoán của 100 động cơ test. Đường chéo ứng với dự đoán hoàn hảo.

6.4 Đối chiếu HGB126 và HGB169 trên panel chung

Để tách ảnh hưởng của biểu diễn đặc trưng khỏi khác biệt về cách chia dữ liệu, alignment audit dùng cùng 17.731 dòng OOF, cùng 100 động cơ và cùng các fold đã lưu. HGB126 có 126 đặc trưng rolling; HGB169 thêm `cycle` và 42 đặc trưng lịch sử từ 30 cycles đầu, tổng cộng 169 đặc trưng. Bảng 17 cho thấy HGB169 tốt hơn trên cả panel trajectory và panel endpoint gồm các mốc 0,5, 0,7 và 0,9 tuổi thọ. Khoảng tin cậy là paired bootstrap theo động cơ, điều kiện trên các dự đoán đã lưu; không bao gồm bất định do fit lại, seed hay chọn mô hình.

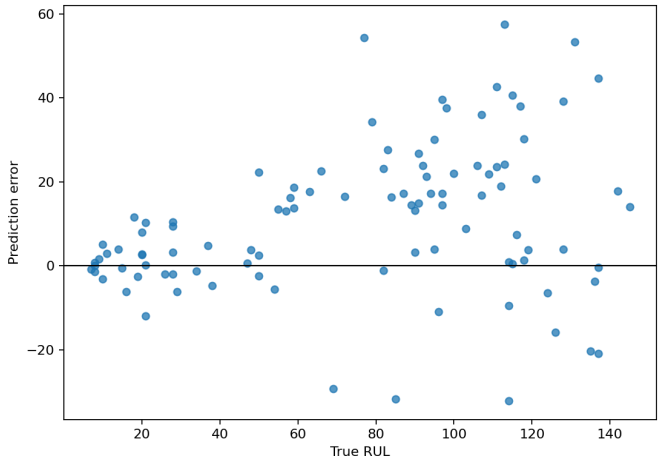
Bảng 17: Metric trên panel chung; delta là HGB169 trừ HGB126.

Panel	HGB126		HGB169	
	MAE	RMSE	MAE	RMSE
Trajectory	20,2143	33,6359	17,0009	29,8068
Endpoint 0,5/0,7/0,9	15,2577	22,3462	9,8913	14,9913

Paired bootstrap cho delta MAE trajectory là $-3,2134$ cycles, khoảng tin cậy 95% $[-4,4586; -1,9220]$. Delta RMSE là $-3,8291$ cycles, khoảng $[-5,5649; -2,0652]$. Ở panel endpoint, delta MAE là $-5,3664$ cycles, khoảng $[-6,9320; -3,8083]$; delta RMSE là $-7,3548$ cycles, khoảng $[-9,7162; -5,0528]$. Theo panel chung này, HGB169 là predictor candidate chính cho phần phân tích; đây là kết luận có điều kiện trên các dự đoán lịch sử, chưa phải xác nhận độc lập bằng một lần fit mù mới.

6.5 Phân tích lỗi

Trên dự đoán test của baseline lịch sử HGB126, sai số tuyệt đối trên 100 động cơ test có trung vị 13,3654 cycles, tứ phân vị 25% là 3,5666 và 75% là 22,3412. Sai số phụ thuộc rõ vào RUL thật (Hình 10): MAE là 4,3830 cycles với 25 động cơ có RUL thật từ 0 đến 30, 8,7734 cycles với nhóm 31–60, và khoảng 21,3–21,5 cycles với các nhóm trên 60. Nói cách khác, mô hình dự đoán tốt khi động cơ đã gần hỏng, tức đúng vùng quan trọng cho cảnh báo, và kém hơn khi động cơ còn khỏe.



Hình 10: Sai số (dự đoán trừ thật) theo RUL thật của 100 động cơ test.

Mô hình có xu hướng dự đoán cao: 74 dự đoán cao hơn RUL thật và 26 thấp hơn. Cả mười sai số lớn nhất đều là dự đoán cao, lệch 37,6–57,5 cycles, và tất cả rơi vào động cơ có RUL thật từ 77 đến 137 cycles (Bảng 18). Vì NASA score phạt dự đoán cao theo hàm mũ, riêng mười động cơ này chiếm khoảng 75,5% tổng NASA score.

Bảng 18: Năm động cơ test có sai số tuyệt đối lớn nhất.

Động cơ	RUL thật	RUL dự đoán	Sai số (dự đoán – thật)
23	113	170,4768	57,4768
67	77	131,2516	54,2516
73	131	184,2864	53,2864
83	137	181,6740	44,6740
22	111	153,5543	42,5543

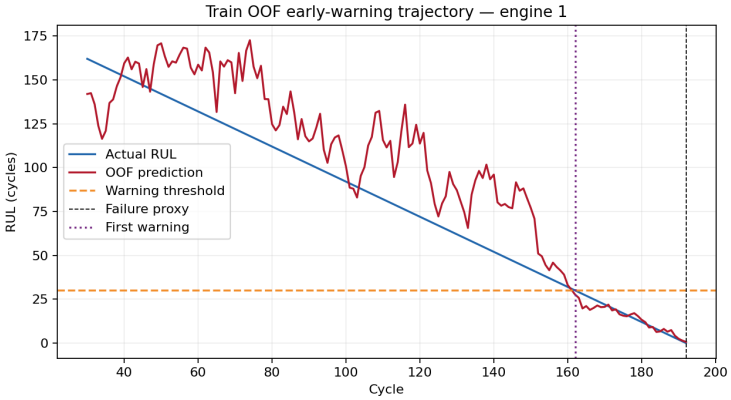
6.6 Cảnh báo sớm

Với ngưỡng RUL dự đoán ≤ 30 cycles trên dự đoán OOF của baseline HGB126, cả 100 động cơ train đều được cảnh báo ít nhất một lần và không động cơ nào bị bỏ lỡ. Tuy nhiên, về thời điểm, 44 động cơ được cảnh báo trước khi RUL thật chạm 30, 7 động cơ đúng lúc và 49 động cơ cảnh báo trễ (nhưng vẫn trước khi hỏng). Lead time có trung vị 30 cycles (Bảng 19); trường hợp trễ nhất chỉ còn 19 cycles trước khi hỏng.

Bảng 19: Phân phối lead time tại cảnh báo đầu tiên trên 100 động cơ train, đơn vị cycle.

Nhỏ nhất	Q1	Trung vị	Trung bình	Q3	Lớn nhất
19	26	30	31,22	35	55

Hình 11 minh họa động cơ train 1, được chọn vì có mã nhỏ nhất. Động cơ này được cảnh báo ở cycle 162 và hỏng ở cycle 192, tức lead time đúng 30 cycles.



Hình 11: RUL thật, dự đoán OOF của baseline HGB126 và ngưỡng 30 cycles của động cơ train 1. Đường đứng màu tím là cảnh báo đầu tiên; đường đứng màu đen là cycle hỏng.

Để tách việc chọn predictor khỏi việc đặt policy cảnh báo, Bảng 20 chỉ mô tả hai ngưỡng cố định trên cùng dự đoán OOF. Threshold 30 cho HGB126 có 51% cảnh báo đúng thời điểm (44 trước và 7 đúng ngưỡng), còn HGB169 có 45%. Threshold 40 tăng tỷ lệ này lên lần lượt 94% và 96%, đồng thời early-alarm burden là 5,329% và 5,553%. Đây là sensitivity analysis; threshold 40 chưa được hiệu chuẩn thành policy triển khai. Historical calibrated169 dùng inner selection theo từng outer fold, có on-time rate 94%, và được giữ như một readout riêng, không trộn với bảng policy cố định.

Bảng 20: Độ nhạy của hai ngưỡng cảnh báo trên cùng dự đoán OOF.

Mô hình	Ngưỡng	Đúng thời điểm	Trễ/bỏ lỡ	Early burden (%)
HGB126	30	51%	49%	1,293
HGB169	30	45%	55%	1,070
HGB126	40	94%	6%	5,329
HGB169	40	96%	4%	5,553

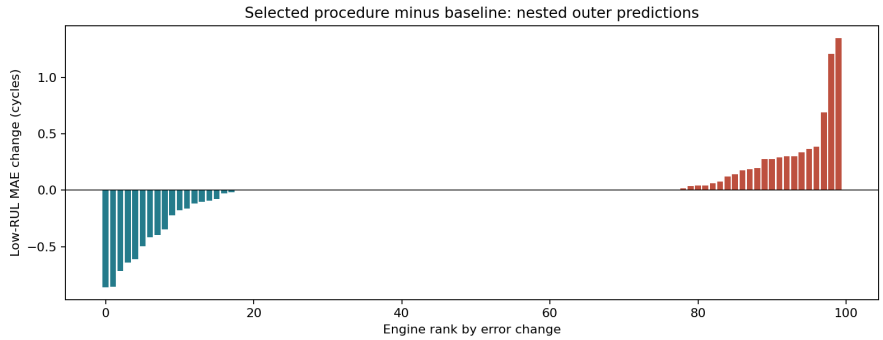
6.7 Thử nghiệm cải thiện bằng nested cross-validation

Thử nghiệm không tìm được cấu hình đáp ứng tiêu chí cải thiện. Ở giai đoạn chọn window, cả 5 vòng ngoài đều chọn lại window 30. Ở giai đoạn chỉnh siêu tham số, vòng ngoài 1 và 3 chọn một cấu hình khác (learning rate 0,03, 400 vòng boosting, 31 lá, tối thiểu 10 mẫu mỗi lá, L2 bằng 0,01), ba vòng còn lại giữ cấu hình gốc. Không vòng nào chọn phương án tăng trọng số. Bảng 21 cho thấy kết quả vòng ngoài sau từng giai đoạn.

Bảng 21: Kết quả vòng ngoài trên 100 động cơ train (17.731 dòng từ cycle 30). Mỗi dòng đánh giá cả quy trình chọn đến hết giai đoạn đó.

Giai đoạn	MAE toàn bộ	MAE RUL ≤ 30	Trễ/bỏ lỡ	Báo sớm (%)
Mô hình gốc	20,2143	3,6676	49/100	1,2933
Sau chọn window	20,2143	3,6676	49/100	1,2933
Sau chỉnh siêu tham số	20,2459	3,6730	53/100	1,2580
Sau chọn trọng số	20,2459	3,6730	53/100	1,2580

So với mô hình gốc, MAE vùng RUL thấp tăng 0,0054 cycles (khoảng tin cậy 95% $[-0{,}0506; 0{,}0639]$) và MAE toàn bộ tăng 0,0315 cycles ($[-0{,}1112; 0{,}1737]$). Cả hai khoảng đều chứa 0, nên không có bằng chứng về thay đổi sai số. Số động cơ cảnh báo trễ tăng từ 49 lên 53 (khoảng tin cậy của mức tăng: 1–8 điểm phần trăm), và trung vị lead time giảm từ 30 xuống 29 cycles. Kết quả theo từng động cơ khá phân tán (Hình 12): ví dụ động cơ 7 giảm MAE vùng RUL thấp từ 4,1811 xuống 3,3209 cycles, còn động cơ 98 tăng từ 3,9296 lên 5,2803 cycles.



Hình 12: Chênh lệch MAE vùng RUL thấp của từng động cơ (quy trình chọn trừ mô hình gốc), sắp theo độ lớn. Giá trị âm là giảm sai số.

Một số cấu hình có giảm sai số vùng RUL thấp nhưng phải đánh đổi ở chỉ số khác (Bảng 22). Cấu hình `hgb_08` giảm MAE vùng RUL thấp khoảng 9,75% nhưng tăng MAE toàn bộ 0,3233 cycles; phương án trọng số $(1; 2; 3)$ giảm cả sai số vùng RUL thấp và số cảnh báo trễ nhưng tăng MAE toàn bộ và tỷ lệ báo sớm. Theo tiêu chí đã chốt, cả hai không được chọn. Các số này là kết quả inner CV dùng để chọn, không phải đánh giá độc lập.

Bảng 22: Ví dụ đánh đổi trong inner CV 3 fold trên toàn bộ train.

Cấu hình	MAE toàn bộ	MAE RUL ≤ 30	Trễ/bỏ lỡ	Báo sớm (%)
Mô hình gốc	20,3613	3,6148	50/100	1,2175
`hgb_08`	20,6846	3,2625	49/100	1,0177
Trọng số $(1; 2; 3)$	20,4072	3,3901	47/100	1,3752

Khi chạy quy trình chọn trên toàn bộ train, cấu hình cuối trùng với mô hình gốc (window 30, cấu hình HGB gốc, trọng số đồng đều). Dự đoán test vì vậy không đổi: MAE 15,3490 cycles, RMSE 20,5413 cycles, và MAE 4,3830 cycles trên 25 động cơ test có RUL thật ≤ 30.

6.8 Đánh giá bổ sung: đối chứng lịch sử và similarity-based RUL

Sau các thí nghiệm ban đầu, một protocol bổ sung được dùng để kiểm tra liệu việc bổ sung thông tin lịch sử hoặc thay đổi cách suy luận có tạo ra cải thiện ổn định hay không. Đối chứng `hgb_baseline169` giữ raw RUL, HGB và window 30 cycles, đồng thời giữ 126 đặc trưng rolling, thêm cột `cycle` và 42 đặc trưng chênh lệch so với 30 cycles đầu (169 đặc trưng tổng cộng). Kết quả outer 5 fold chia theo động cơ được so với hai ablation: thêm thống kê window 10 và thêm profile đầu vòng đời. Promotion gate yêu cầu đồng thời cải thiện trajectory RMSE, không xấu endpoint MAE/NASA, on-time và early-alarm burden.

Thêm window 10 chỉ giảm trajectory RMSE điểm (29,4802 so với 29,8068 của HGB) nhưng làm endpoint MAE tăng từ 9,8913 lên 9,9876, NASA từ 6,8838 lên 7,5664 và on-time giảm từ 0,94 xuống 0,91; khoảng tin cậy 95% của chênh lệch RMSE là $[-0,6886; 0,0596]$. Profile 30 cycles đầu còn đạt RMSE 33,1280, endpoint MAE 13,1723 và NASA 41,0964. Vì vậy outer selection giữ HGB baseline169 ở cả năm fold. Kết quả này được dùng cùng alignment audit để xác định HGB169 là predictor candidate chính có điều kiện; HGB126 vẫn được giữ như baseline lịch sử vì chưa có blind refit độc lập cho HGB169.

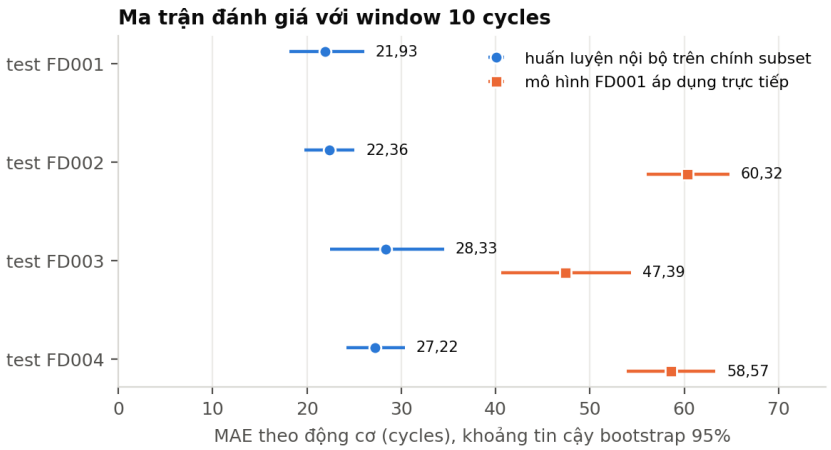
Một pilot CPU nhỏ tiếp tục kiểm tra *similarity-based RUL*, trong đó mỗi đoạn sensor 30 cycles được đối chiếu với nhiều engine train đã chạy đến hỏng theo ý tưởng thư viện quỹ đạo suy giảm [20]. Pilot dùng 3 fold theo engine, hai cấu hình $k \in \{5, 10\}$ và không đọc nhãn test. HGB 169 đạt trajectory RMSE 28,8350 trên panel pilot; similarity $k = 5$ và $k = 10$ đạt lần lượt 37,1730 và 36,5430, đồng thời endpoint MAE và burden đều xấu hơn. Cả hai cấu hình thua HGB169 ở 3/3 fold, nên nhánh này được dừng và không mở full nested study.

6.9 Chuyển miền sang FD002–FD004

Bảng 23 và Hình 13 trình bày ma trận đánh giá với window 10 cycles. Khi mỗi subset tự huấn luyện và đánh giá, MAE nằm trong khoảng 21,93–28,33 cycles. Khi mô hình FD001 được áp dụng trực tiếp, MAE tăng lên 60,32 cycles trên FD002, 47,39 trên FD003 và 58,57 trên FD004. Khoảng tin cậy của MAE chuyển miền và MAE nội bộ trên cùng subset không chồng nhau ở cả ba trường hợp.

Bảng 23: Ma trận đánh giá nội bộ và chuyển miền, window 10 cycles. Khoảng tin cậy 95% của MAE lấy mẫu lại động cơ 10.000 lần.

Huấn luyện → Test	Loại	MAE	Khoảng tin cậy MAE	RMSE	NASA score
FD001 → FD001	Nội bộ	21,9258	[18,1054; 26,0412]	29,5936	8.103,0148
FD002 → FD002	Nội bộ	22,3556	[19,7104; 25,0450]	31,3077	34.004,9782
FD003 → FD003	Nội bộ	28,3255	[22,4736; 34,5537]	41,7606	4.109.107,7036
FD004 → FD004	Nội bộ	27,2218	[24,1806; 30,3734]	36,9311	492.703,0709
FD001 → FD002	Chuyển miền	60,3215	[56,0478; 64,7966]	70,4603	14.193.434,0842
FD001 → FD003	Chuyển miền	47,3949	[40,6269; 54,3524]	59,2140	59.380.624,2646
FD001 → FD004	Chuyển miền	58,5731	[53,9262; 63,3042]	69,8148	14.691.764,3175



Hình 13: MAE và khoảng tin cậy 95% của ma trận chuyển miền. Trên cả ba subset đích, khoảng tin cậy của mô hình FD001 (vuông cam) không chồng với khoảng tin cậy của mô hình huấn luyện nội bộ (tròn xanh).

MAE nội bộ của FD001 với window 10 (21,93 cycles) cao hơn MAE 15,35 cycles với window 30. Kết quả này phù hợp với nhận định rằng window dài hơn mang nhiều thông tin xu hướng hơn, nhưng hai cấu hình chưa được so sánh trong cùng một thí nghiệm có kiểm soát.

6.10 Health Index và dự báo ARIMA

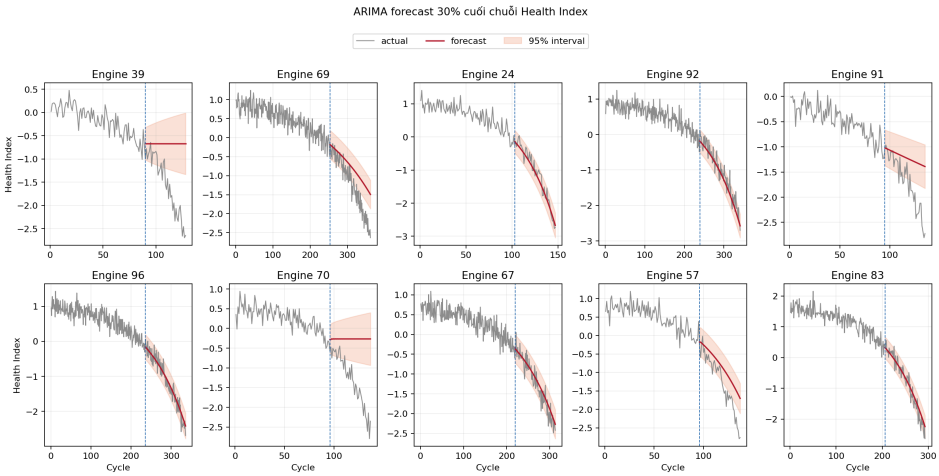
Cả 10 mô hình ARIMA đều hội tụ; tổng cộng có 704 điểm dự báo trên đoạn 30% cuối. Bảng 24 cho thấy chất lượng dự báo khác nhau nhiều giữa các động cơ: MAE từ 0,1515 đến 1,0372 và tỷ lệ bao phủ của khoảng dự đoán 95% từ 21,43% đến 97,03%. Năm động cơ có tỷ lệ bao phủ dưới 51%, tức khoảng dự đoán hẹp hơn nhiều so với biến động thực tế.

Bảng 24: Kết quả dự báo Health Index trên 30% cuối chuỗi của 10 động cơ.

Động cơ	Fold	Bậc	MAE	RMSE	Bao phủ 95%
39	1	(0,1,1)	0,8860	1,0788	35,90%
69	1	(1,0,1)	0,4585	0,5645	50,46%
24	2	(1,0,1)	0,1572	0,1897	95,56%
92	2	(1,0,1)	0,1631	0,2066	89,32%
91	3	(1,0,1)	0,4900	0,6422	48,78%
96	3	(1,0,1)	0,1515	0,1832	97,03%
70	4	(2,1,2)	1,0372	1,2205	21,43%
67	4	(2,0,1)	0,1607	0,2018	93,62%
57	5	(1,0,2)	0,4346	0,5230	50,00%
83	5	(1,0,1)	0,1708	0,2171	87,50%

Tính trung bình trên 10 động cơ, ARIMA có MAE 0,4110 và RMSE 0,5027, thấp hơn baseline `drift` (0,7469 và 0,9128) và `last_value` (0,9478 và 1,1553). Tuy vậy, ARIMA chỉ tốt hơn `drift` ở 7/10 động cơ và tốt hơn `last_value` ở 8/10 động cơ. Với 10 động cơ, đây là so sánh mô tả, chưa đủ để kết luận ARIMA tốt hơn một cách tổng quát.

Về chẩn đoán, ADF không bác bỏ giả thuyết có nghiệm đơn vị ở cả 10 đoạn fit (p-value 0,8606–0,9990), tức không có bằng chứng các chuỗi là dừng. Ljung–Box có p-value dưới 0,05 ở động cơ 69 và 70, cho thấy phần dư của hai mô hình này vẫn còn tự tương quan. Hình 14 minh họa dự báo và khoảng dự đoán.



Hình 14: Health Index thực tế, dự báo ARIMA và khoảng dự đoán 95% trên 30% cuối chuỗi của 10 động cơ.

6.11 ETL, parity và benchmark

ETL tạo lớp Silver gồm 33.727 quan sát train/test và 20.631 nhãn train, cùng lớp Gold gồm 200 dòng engine summary trong Parquet và SQLite. Mọi bảng được đọc lại sau khi ghi để kiểm tra. Trên cả ba bảng, kết quả PySpark khớp hoàn toàn với Pandas về schema, khóa, số dòng và giá trị; sai khác tuyệt đối lớn nhất bằng 0.

Bảng 25 cho thấy thời gian Pandas tạo engine summary tăng gần tuyến tính theo kích thước dữ liệu, từ khoảng 0,02 giây với 33.727 dòng đến khoảng 1,67 giây với 3.372.700 dòng. Mỗi mức chỉ được đo hai lần liên tiếp trong cùng tiến trình, nên chưa đủ để ước lượng độ biến thiên của thời gian chạy.

Bảng 25: Thời gian Pandas tạo engine summary trên dữ liệu nhân bản trong bộ nhớ.

Hệ số nhân	Dòng đầu vào	Dòng đầu ra	Lần 1 (s)	Lần 2 (s)
1×	33.727	200	0,0195	0,0196
10×	337.270	2.000	0,1589	0,1474
100×	3.372.700	20.000	1,6752	1,6732

Kết quả được trình bày thêm qua một dashboard Streamlit chỉ đọc các file đã ghi: người dùng có thể xem metric, chọn một động cơ test để so sánh RUL thật và dự đoán, hoặc chọn một trong 10 động cơ để xem dự báo Health Index. Dashboard không huấn luyện lại mô hình và không đọc file nhãn test gốc.

Bảng trên là benchmark ETL lịch sử, chỉ đo phép tổng hợp `build_engine_summary` bằng Pandas trong bộ nhớ. Benchmark bổ sung có đọc/ghi, rolling features và đối chiếu Pandas–PySpark ở chế độ local được trình bày riêng trong Mục 7.7; hai phép đo dùng protocol và mục đích khác nhau nên không được gộp thành một con số hiệu năng.

7 Thảo luận

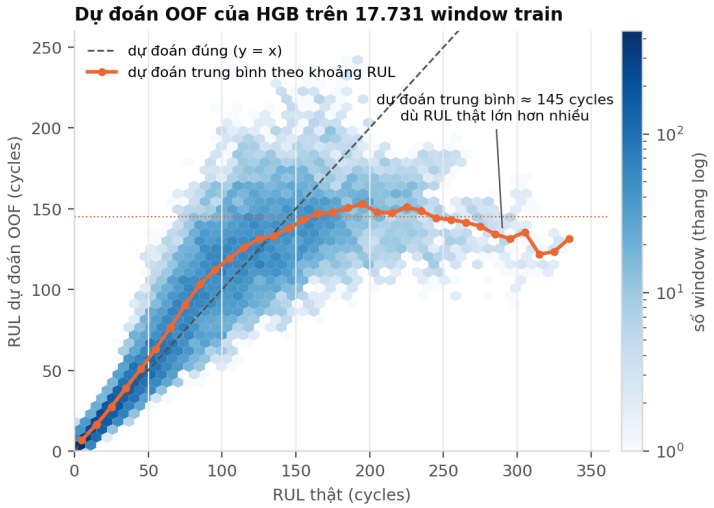
Mục này giải thích các kết quả ở Mục 6: sai số của mô hình RUL, thời điểm cảnh báo, sự khác biệt giữa hai biểu diễn HGB126/HGB169 và khả năng chuyển miền sang các subset khác. Các số liệu chẩn đoán dùng trong mục này được tính lại từ artifact đã có bằng `scripts/report_error_diagnostics.py`, không huấn luyện lại mô hình.

7.1 Sai số của mô hình RUL đến từ đâu

Bảng 26 và Hình 15 chia sai số OOF của HGB126 trên train theo RUL thật. Mô hình dự đoán tốt ở cuối vòng đời: MAE chỉ 3,67 cycles khi RUL thật không quá 30. Sai số tăng dần khi RUL lớn hơn, và thay đổi hẳn tính chất khi RUL vượt khoảng 150 cycles. Ở vùng này, dự đoán trung bình gần như không đổi, quanh 145–147 cycles, bất kể RUL thật là 160 hay 300. Nói cách khác, mô hình đã tự học ra một "trần" khoảng 145 cycles, và mọi động cơ còn xa thời điểm hỏng đều bị dự đoán gần mức đó.

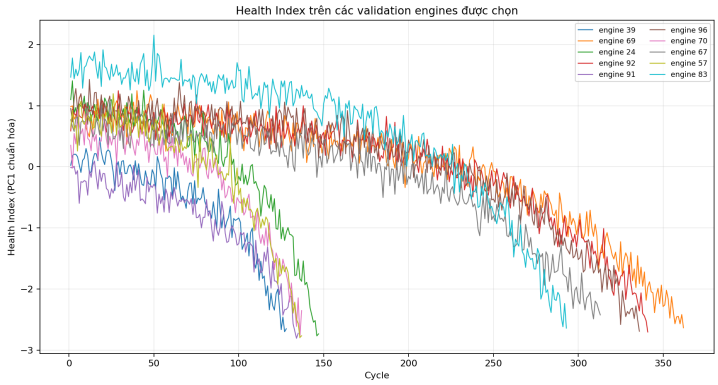
Bảng 26: Sai số OOF của HGB trên 17.731 window train, chia theo RUL thật. Sai số có dấu bằng dự đoán trừ thật.

RUL thật (cycles)	Số window	MAE	Sai số có dấu trung bình	Dự đoán trung bình
0–30	3.100	3,67	+1,47	16,47
31–60	3.000	10,68	+5,59	51,09
61–90	3.000	20,85	+14,52	90,02
91–125	3.392	22,81	+12,88	120,61
126–150	1.999	20,82	−2,87	134,73
151–200	2.226	30,25	−23,30	147,16
201–361	1.014	91,29	−91,29	145,36



Hình 15: RUL dự đoán OOF so với RUL thật. Đường cam là dự đoán trung bình theo từng khoảng 10 cycles: bám sát đường chéo khi RUL nhỏ, nhưng dừng quanh 145 cycles khi RUL thật lớn hơn khoảng 150.

Hiện tượng này phù hợp với hình dạng của tín hiệu. Hình 16 cho thấy Health Index của các động cơ gần như đi ngang, chỉ dao động quanh một mức, trong phần lớn vòng đời và chỉ giảm rõ ở giai đoạn cuối. Khi động cơ còn khỏe, sensor chủ yếu phản ánh dao động đo và khác biệt ban đầu giữa các động cơ, nên khó phân biệt một động cơ còn 160 cycles với một động cơ còn 300 cycles. Kết quả này phù hợp với giả thuyết rằng biểu diễn hiện tại chứa ít tín hiệu phân biệt RUL xa; dữ liệu hiện có chưa đủ để khẳng định một giới hạn thông tin cố định. Nó cũng giải thích vì sao MAE cross-validation (20,21 cycles) cao hơn MAE test (15,35 cycles): 29,5% số window train có RUL thật trên 125, trong khi RUL thật của test chỉ từ 7 đến 145.



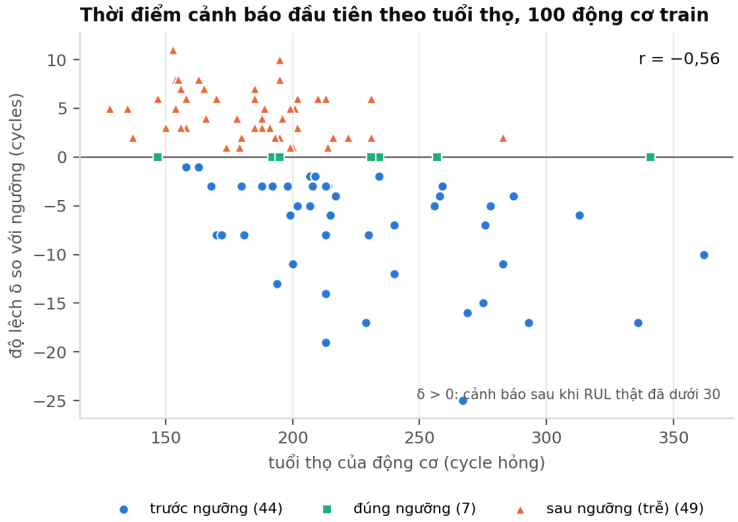
Hình 16: Health Index theo cycle của 10 động cơ train được chọn ở Mục 5.5. Chỉ số gần như đi ngang ở phần lớn vòng đời và giảm rõ ở giai đoạn cuối.

Ở vùng giữa (RUL 61–125), mô hình có xu hướng dự đoán cao hơn thật khoảng 13–15 cycles. Mười sai số lớn nhất trên test rơi vào các động cơ có RUL thật 77–137, phần lớn trùng với vùng này, và đều là dự đoán cao; riêng mười trường hợp này chiếm 75,5% NASA score. Như vậy, sai số test không phải ngẫu nhiên mà lặp lại đúng xu hướng đã thấy trên OOF.

So với nghiên cứu liên quan, Li và cộng sự báo cáo RMSE 12,61 cycles trên FD001 với CNN khi cắt nhãn ở 125 cycles, và 13,32 cycles khi không cắt nhãn [7]. RMSE của báo cáo là 20,54 cycles. Hai kết quả không so sánh trực tiếp được, vì Li và cộng sự dùng 14 sensor, cách chuẩn hóa và mô hình khác. Tuy vậy, khoảng cách khoảng 7 cycles so với cả phương án không cắt nhãn cho thấy việc chỉ cắt nhãn có thể chưa đủ: sáu thống kê trên window 30 cycles có thể đã làm mất một phần thông tin về hình dạng chuỗi mà CNN khai thác được.

7.2 Vì sao gần một nửa cảnh báo bị trễ

Cảnh báo trễ không phân bố đều giữa các động cơ. 49 động cơ bị cảnh báo sau ngưỡng có tuổi thọ trung bình 182,8 cycles, trong khi 44 động cơ được cảnh báo trước ngưỡng có tuổi thọ trung bình 229,1 cycles. Hệ số tương quan giữa tuổi thọ và độ lệch cảnh báo δ_i là $-0{,}56$: động cơ càng ngắn đời thì càng bị cảnh báo muộn (Hình 17).



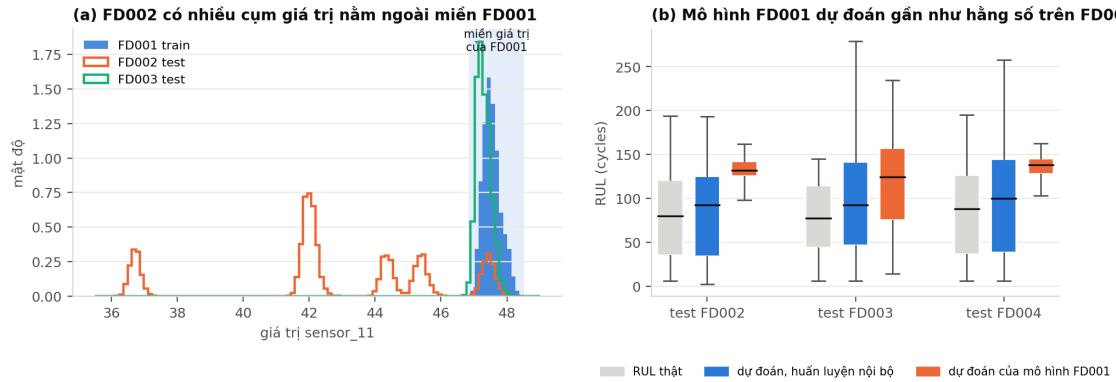
Hình 17: Độ lệch của cảnh báo đầu tiên so với ngưỡng theo tuổi thọ của 100 động cơ train. Cảnh báo trễ (tam giác cam) tập trung ở các động cơ hỏng trước khoảng cycle 230.

Một cách giải thích là mô hình học một quan hệ "trung bình" giữa mức sensor và RUL. Động cơ ngắn đời suy giảm nhanh hơn trung bình, nên tại cùng một mức sensor, RUL thật của chúng thấp hơn điều mô hình kỳ vọng. Kết quả là RUL dự đoán giảm xuống dưới 30 muộn hơn thực tế. Điều này nhất quán với sai số dương ở vùng RUL 31–90 trong Bảng 26. Đặc trưng độ dốc trong window 30 cycles lẽ ra phải phản ánh tốc độ suy giảm, nhưng trên 30 cycles, xu hướng thật có thể bị dao động ngắn hạn che khuất. Phân tích trailing mean chỉ cho thấy biến động được làm dịu trong mô tả tín hiệu; nó không chứng minh dữ liệu có nhiễu đo đã biết hoặc rằng smoothing chắc chắn cải thiện RUL.

Từ góc độ ứng dụng, đây là loại lỗi nguy hiểm hơn cả: động cơ hỏng nhanh nhất lại là động cơ nhận cảnh báo muộn nhất. Một ngưỡng cố định 30 cycles vì vậy chưa đủ. Muốn bảo đảm lead time tối thiểu, ngưỡng cần có biên an toàn, hoặc mô hình cần biết được tốc độ suy giảm riêng của từng động cơ. Cũng cần nhớ rằng các con số cảnh báo được tính trên cùng tập OOF đã dùng để chọn mô hình, nên trên dữ liệu mới, tỷ lệ cảnh báo trễ có thể còn cao hơn.

7.3 Vì sao mô hình FD001 không chuyển được sang subset khác

Bảng 27 và Hình 18 cho thấy mô hình FD001 thất bại theo hai kiểu khác nhau.



Hình 18: (a) Phân phối `sensor_11`: FD003 nằm trong miền giá trị của FD001, còn FD002 tách thành nhiều cụm theo điều kiện vận hành, phần lớn nằm ngoài miền này (FD004 có dạng tương tự FD002). (b) Phân phối RUL thật và RUL dự đoán trên test của từng subset: mô hình FD001 cho dự đoán rất hẹp trên FD002 và FD004.

Bảng 27: Chẩn đoán dự đoán chuyển miền (window 10). "Trong miền FD001" là tỷ lệ trung bình giá trị test của 9 sensor có xu hướng nằm trong khoảng min–max của train FD001. TB: trung bình.

Đánh giá	Trong miền FD001	Sai số có dấu TB	Tỷ lệ dự đoán cao	Độ lệch chuẩn dự đoán	RUL thật TB
FD001 → FD002	15,2%	+50,79	81%	13,33	81,19
FD001 → FD003	94,7%	+45,34	83%	55,01	75,32
FD001 → FD004	14,1%	+49,39	79%	12,72	86,55
FD002 → FD002	–	+2,64	54%	52,53	81,19
FD003 → FD003	–	+22,85	75%	63,11	75,32
FD004 → FD004	–	+12,18	62%	62,59	86,55

Với FD002 và FD004, chỉ khoảng 14–15% giá trị sensor của test nằm trong miền giá trị mà mô hình đã thấy khi huấn luyện, do sáu điều kiện vận hành đẩy mức sensor ra xa điều kiện duy nhất của FD001. Mô hình cây không ngoại suy được: mọi giá trị nằm ngoài miền huấn luyện đều rơi vào các lá ở biên. Hệ quả là dự đoán gần như hằng số (độ lệch chuẩn khoảng 13 cycles, so với khoảng 52–63 khi huấn luyện nội bộ) và cao hơn RUL thật khoảng 50 cycles. Như vậy, thất bại ở đây nhiều khả năng đến từ dịch chuyển phân bố đầu vào (covariate shift) do điều kiện vận hành, hơn là từ việc mô hình hiểu sai quá trình suy giảm. Khi tự huấn luyện trên FD002, mô hình gần như không lệch (sai số có dấu $+2{,}64$), cho thấy thông tin để dự đoán vẫn có sẵn trong dữ liệu.

FD003 là trường hợp khác. Giá trị sensor gần như nằm hoàn toàn trong miền FD001 (94,7%) và dự đoán vẫn biến thiên (độ lệch chuẩn 55), tức mô hình có "phản ứng" với dữ liệu. Tuy nhiên, dự đoán vẫn cao hơn thật trung bình 45 cycles. FD003 có thêm dạng lỗi fan; một giả thuyết hợp lý là dạng lỗi này làm sensor thay đổi theo kiểu khác với suy giảm HPC mà mô hình đã học, nên động cơ trông "khỏe" hơn thực tế. Ngay cả khi huấn luyện nội bộ, FD003 vẫn bị dự đoán cao trung bình 22,85 cycles, cho thấy hai dạng lỗi trong cùng một tập là khó hơn cho cách biểu diễn hiện tại.

Hai kiểu thất bại này cần hai cách xử lý khác nhau: FD002 và FD004 cần loại bỏ ảnh hưởng của điều kiện vận hành trước khi tạo đặc trưng; FD003 cần dữ liệu huấn luyện có dạng lỗi tương ứng hoặc đặc trưng phân biệt được dạng lỗi.

7.4 Vì sao thử nghiệm cải thiện không thành công

Các hướng đã thử trong nested CV (đổi window, chỉnh siêu tham số và tăng trọng số vùng RUL thấp) chưa tìm được cấu hình mới vượt bộ tiêu chí đã chốt. Đợt HGB bổ sung cho thấy thêm window 10 có thể giảm RMSE trajectory nhưng làm xấu endpoint hoặc on-time; profile 30 cycles đầu còn làm RMSE tăng. Pilot similarity-based RUL cũng không đạt: RMSE 36,543–37,173 so với 28,835 của HGB 169 trên cùng panel 3 fold, và bị dừng trước full study. Các kết quả này không chứng minh rằng mọi kiến trúc mới đều kém; chúng chỉ cho thấy các thay đổi đã được kiểm tra chưa giải quyết được hai nguyên nhân sai số chính: tín hiệu đầu vòng đời khó phân biệt RUL xa và tốc độ suy giảm khác nhau giữa các động cơ.

Alignment audit cho thấy HGB169 tốt hơn HGB126 trên panel trajectory và endpoint chung, nên HGB169 được chọn làm predictor candidate chính trong phạm vi bằng chứng hiện có. Kết luận này có điều kiện: so sánh dựa trên dự đoán đã lưu, bootstrap không bao gồm bất định fit lại, và run lịch sử đã được kiểm tra trước; chưa có một lần refit mù độc lập để xác nhận. Vì vậy metric test của HGB126 vẫn được giữ như baseline lịch sử; báo cáo không dùng metric test của HGB169 để chọn predictor.

7.5 Ý nghĩa đối với xử lý dữ liệu lớn

Với 265.256 quan sát trên cả bốn subset, dữ liệu C-MAPSS vẫn vừa trong bộ nhớ của một máy, nên Pandas đủ dùng. Giá trị của phần dữ liệu lớn trong báo cáo nằm ở cách tổ chức: kiểm tra contract trước khi xử lý, lưu dữ liệu theo lớp, và có một implementation ETL bằng PySpark cho kết quả giống hệt Pandas. Nhờ đó, khi dữ liệu tăng lên (nhiều đội động cơ, tần suất lấy mẫu cao hơn), phần ETL có thể chuyển sang Spark với bằng chứng parity sẵn có. Đợt bổ sung đã triển khai sáu thống kê cho 21 sensor bằng Spark và kiểm tra toàn bộ rolling features ở scale $1\times$. Benchmark local có I/O được trình bày ở Bảng 30.

Để tránh đồng nhất Big Data với riêng kích thước tập tin, Bảng 28 đối chiếu năm khía cạnh của bài toán với bằng chứng thực tế và giới hạn tương ứng. Đây là pipeline batch theo hướng Big Data; báo cáo không gán cho C-MAPSS hiện tại quy mô phân tán mà dữ liệu chưa có.

Bảng 28: Đối chiếu 5V, bằng chứng và giới hạn của pipeline.

Khía cạnh	Bằng chứng trong báo cáo	Giới hạn hiện tại
Volume	265.256 quan sát từ bốn subset; dữ liệu raw khoảng 46 MB; kích thước bảng được ghi ở từng bước ETL.	Vẫn vừa bộ nhớ một máy; benchmark nhân bản chỉ đo chi phí kỹ thuật.
Velocity	Pipeline xử lý batch theo từng subset, có thể chạy lại từ file raw và Parquet.	Chưa có streaming, ingest liên tục hoặc yêu cầu độ trễ thời gian thực.
Variety	Có 21 sensor, 3 operational settings, khóa engine/cycle, chuỗi run-to-failure và nhãn RUL; bốn subset thể hiện các tổ hợp điều kiện vận hành và dạng lỗi khác nhau, được lưu qua text, Parquet, SQLite và bảng đặc trưng.	Các định dạng lưu trữ vẫn xuất phát từ cùng nguồn cảm biến dạng bảng; chưa có nhiều nguồn dị thể hoặc dữ liệu phi cấu trúc.
Veracity	Contract schema, kiểm tra khóa và cycle, kiểm tra giá trị thiếu, hash và parity Pandas–PySpark.	Chưa kiểm chứng với lỗi cảm biến và dữ liệu thiếu của hệ thống thực.
Value	RUL, cảnh báo sớm và đánh giá chuyển miền cho FD001–FD004.	Chưa có chi phí bảo trì, downtime hoặc KPI kinh tế.

7.6 Giới hạn

Về dữ liệu. C-MAPSS là dữ liệu mô phỏng; động cơ thực tế có lỗi cảm biến, dữ liệu thiếu và điều kiện vận hành phức tạp hơn. Dữ liệu cũng không có chi phí bảo trì hay cách quy đổi cycle sang thời gian thực, nên báo cáo không thể đánh giá giá trị kinh tế của cảnh báo.

Về đánh giá. Metric test chỉ đo tại một điểm cắt mỗi động cơ, trong khi trong vận hành, mô hình sẽ được gọi ở mọi cycle. Kết quả cảnh báo dùng lại tập OOF đã dùng để chọn mô hình. Nhãn test FD001 đã được xem trước thử nghiệm nested CV, nên test không còn hoàn toàn độc lập với thử nghiệm đó. So sánh giữa window 10 và window 30 ở Mục 6.9 không phải một thí nghiệm có kiểm soát. Các giải thích trong Mục 7.1–7.3 là giả thuyết phù hợp với dữ liệu; thí nghiệm bổ sung dưới đây kiểm tra nhãn, lịch sử và điều kiện vận hành nhưng chưa chứng minh cơ chế nhân quả.

Về hạ tầng. Benchmark bổ sung đo Pandas và PySpark local, gồm đọc ghi file. Spark ghi qua Arrow trên driver trong môi trường Windows; chưa đo distributed writer hoặc cụm nhiều máy. Dữ liệu nhân bản không làm tăng số mẫu khoa học độc lập. Việc công bố nhiều file qua staging và rollback không phải atomic transaction (giao dịch nguyên tử): chưa bảo đảm đọc/ghi đồng thời hoặc mất điện đột ngột. Đợt kiểm chứng mã nguồn ngày 07/10/2026 chạy trên Windows; Linux mới được đối chiếu dependency theo nền tảng và phiên bản từ lock, chưa chạy trực tiếp trong đợt này.

7.7 Thí nghiệm bổ sung và hướng phát triển

Ngày 29/09/2026, ba nghiên cứu nested GroupKFold 5×3 được chạy theo protocol ghi trước khi fit. FD001 dùng window 30, kết hợp cap $R_{\max} \in \{110, 125, 140, \infty\}$ với bật/tắt 42 history features: độ lệch so với mean 30 cycles đầu và slope trên toàn bộ prefix đã quan sát. Ví dụ, feature ở cycle 40 không dùng cycle 41 trở đi. Chỉ cắt nhãn train, giữ nguyên nhãn validation/test. HGB giữ cấu hình baseline và tổng trọng số mỗi engine bằng 1.

FD002/FD004 dùng window 10, so baseline với thêm settings và chuẩn hóa sensor theo sáu cụm điều kiện vận hành. Scaler, KMeans và mean/std theo cụm đều chỉ fit trên training engines của từng fold. Đây là đánh giá trong từng subset, không phải chuyển trực tiếp từ FD001. FD001 yêu cầu CI 95% của delta low-RUL MAE và late rate đều dưới 0, overall MAE/burden không tăng; hai subset còn lại yêu cầu upper CI delta overall MAE dưới 0. Bootstrap dùng 10.000 lần lấy lại engine, có điều kiện theo dự đoán đã fit. Test lịch sử đã được xem, nên đây là follow-up khám phá; test không quyết định lựa chọn. Chi tiết cấu hình, CI và từng metric được lưu tại `experiments/FOLLOWUP_20260929.md`.

Bảng 29: Follow-up nested CV: MAE vòng ngoài và test (cycles). Tiêu chí được chốt riêng cho từng nghiên cứu.

Subset	Outer baseline	Outer chọn	Test chọn	Đạt tiêu chí
FD001	20,21	20,30	13,05	Không
FD002	29,29	28,04	19,18	Có
FD004	36,74	35,45	26,61	Có

FD001: MAE RUL thấp 3,67 chuyển thành 3,22 cycles; cảnh báo trễ/bỏ lỡ 49 chuyển thành 55/100. Test NASA score tăng từ 1513,26 lên 2605,28, dù MAE giảm. Vì vậy giữ baseline mặc định.

Bảng 30: Benchmark local, trung vị ba lượt; thời gian giây, RSS gồm Python và JVM. Cột Pipeline không gồm startup; tổng thời gian gồm startup được lưu trong artifact.

Engine	Scale	Đọc	Feature	Ghi	Pipeline	RSS (GiB)
pandas	1	0,05	10,30	0,15	10,54	0,25
pandas	10	0,09	110,39	0,97	111,68	1,73
spark	1	4,88	7,03	1,43	14,60	1,48
spark	10	5,28	22,12	4,08	32,68	3,05

Tiêu chí FD002/FD004 chỉ xác nhận giảm overall MAE. Cảnh báo trễ chưa giảm và outer NASA score FD004 xấu hơn; chưa đủ bằng chứng cải thiện toàn diện cho bảo trì.

Benchmark dùng cùng Parquet input, FD001 1×/10×, ba lượt với process riêng cho từng lượt, Spark local[4], heap 4 GiB và bốn partition. Pipeline gồm đọc, tổng hợp, rolling và ghi; startup được báo riêng trong artifact và không nằm trong cột Pipeline. Spark write time gồm cả chuyển Arrow về driver. Peak RSS lấy mẫu mỗi 50 ms gồm Python và JVM; không xóa OS cache. Ba lượt scale 1× đều khớp toàn bộ 27.927 dòng feature với tolerance 10^{-7}.

Ở benchmark local, khi scale FD001 từ $1\times$ lên $10\times$, thời gian các stage sau startup của Pandas tăng từ 10,54 lên 111,68 giây, còn Spark tăng từ 14,60 lên 32,68 giây. Nếu tính cả startup, tổng thời gian Spark tương ứng là 20,07 và 38,11 giây; startup không được gộp vào cột Pipeline. Kết quả này chỉ cho thấy chi phí trong resource envelope đã đo và lý do cần giữ implementation Spark khi dữ liệu tăng, không phải bằng chứng Spark đã nhanh hơn trên cluster: phép đo dùng Spark local[4] và ghi qua driver.

Đánh giá cảnh báo đầy đủ hơn. Thay vì ngưỡng cố định 30 cycles, có thể chọn ngưỡng sao cho lead time đạt một mức tối thiểu với xác suất cho trước, dựa trên phân phối sai số OOF ở vùng RUL thấp, rồi kiểm tra trên vòng ngoài của nested CV. Các metric theo toàn bộ quỹ đạo như prognostic horizon và α–λ accuracy [10] cho biết mô hình bắt đầu dự đoán đủ chính xác từ thời điểm nào, điều mà metric tại một điểm cắt không cho biết.

Mô hình học trực tiếp từ chuỗi và cluster. Khoảng cách RMSE so với CNN [7] gợi ý rằng sáu thống kê trên window có thể chưa đủ. Một mô hình CNN một chiều hoặc LSTM [22] nên được so sánh với HGB trên cùng các fold và cùng cách xử lý nhãn, kèm theo chi phí huấn luyện, để biết phần cải thiện đến từ mô hình hay từ cách xử lý nhãn. Phần hạ tầng cần đo thêm distributed writer và pipeline trên cụm máy thực trước khi kết luận về mở rộng.

8 Kết luận

Báo cáo đã xây dựng một pipeline tái lập được để dự đoán RUL trên NASA C-MAPSS, từ dữ liệu thô qua các lớp Bronze–Silver–Gold đến mô hình và đánh giá, trong đó nhãn test được tách khỏi huấn luyện và chọn cấu hình. Các câu hỏi nghiên cứu được trả lời như sau:

Q1. Dữ liệu FD001 (20.631 dòng train, 13.096 dòng test) đạt mọi kiểm tra về schema, khóa và tính liên tục của cycle; sáu sensor hằng số và không mang thông tin.

Q2. 15/21 sensor thay đổi có ý nghĩa thống kê giữa 30 cycles đầu và 30 cycles cuối vòng đời; năm sensor 4, 7, 11, 12 và 20 thay đổi cùng hướng ở cả 100 động cơ.

Q3. ETL bằng PySpark khớp hoàn toàn với Pandas trên ba bảng Silver/Gold.

Q4. Protocol lịch sử với HGB126 và 126 đặc trưng từ window 30 cycles đạt MAE 15,3490 cycles (khoảng tin cậy 95%: 12,7286–18,0866) và RMSE 20,5413 cycles trên 100 động cơ test. Khi chấm lại HGB126 và HGB169 trên cùng 17.731 dòng OOF và cùng các fold, HGB169 (thêm `cycle` và 42 đặc trưng lịch sử) đạt MAE trajectory 17,0009 và RMSE 29,8068, so với 20,2143 và 33,6359 của HGB126; trên panel endpoint tương ứng là 9,8913 và 14,9913 so với 15,2577 và 22,3462. Vì vậy HGB169 là predictor candidate chính theo bằng chứng panel chung. Đây là alignment audit trên dự đoán đã lưu, chưa phải xác nhận độc lập bằng refit mù; metric test HGB126 vẫn được giữ như baseline lịch sử.

Q5. Với cảnh báo, threshold 30 cycles là sensitivity policy cố định: tỷ lệ đúng thời điểm là 51% với HGB126 và 45% với HGB169; threshold 40 nâng các tỷ lệ này lên 94% và 96%, với early-alarm burden lần lượt 5,329% và 5,553%. Các số liệu này được tính trên OOF và chưa phải policy bảo trì đã hiệu chuẩn.

Q6. Mô hình FD001 không dùng trực tiếp được cho các subset khác: MAE tăng từ 21,93–28,33 cycles (huấn luyện nội bộ) lên 47,39–60,32 cycles (chuyển miền). Với FD002 và FD004, phần lớn giá trị sensor nằm ngoài miền huấn luyện nên dự đoán gần như hằng số; với FD003, dự đoán bị lệch cao dù sensor nằm trong miền quen thuộc.

Nhìn chung, pipeline cho kết quả RUL hợp lý trên FD001 và có ranh giới dữ liệu rõ ràng. Phân tích lỗi chỉ ra ba hạn chế có nguyên nhân cụ thể: sai số lớn ở đầu vòng đời do tín hiệu sensor chưa thay đổi, cảnh báo trễ ở động cơ suy giảm nhanh, và mất khả năng dự đoán khi điều kiện vận hành thay đổi. Các ablation thêm window 10, profile đầu vòng đời và similarity retrieval chưa vượt promotion gate của HGB169; điều đó không chứng minh mọi kiến trúc mới đều kém. HGB169 được giữ là predictor candidate chính với giới hạn alignment audit và historical inspection đã nêu. Hướng phát triển phù hợp là refit mù độc lập, hiệu chuẩn policy cảnh báo và đánh giá sai số theo toàn quỹ đạo.

Tài liệu tham khảo

[1] Apache Software Foundation. Apache Parquet Documentation: Overview. Apache Parquet documentation. Accessed 2026-10-07. URL: https://parquet.apache.org/docs/overview/.

[2] Apache Software Foundation. Spark SQL and DataFrames. Apache Spark documentation. Accessed 2026-09-22; implementation version not selected. URL: https://spark.apache.org/docs/latest/sql-programming-guide.html.

[3] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi:10.1111/j.2517-6161.1995.tb02031.x.

[4] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi:10.1214/aos/1176344552.

[5] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3 edition, 2021. Online edition, sections 5.4, 9.1, 9.5 and 9.7; accessed 2026-09-22. URL: https://otexts.com/fpp3/.

[6] Yaguo Lei, Naipeng Li, Liang Guo, Ningbo Li, Tao Yan, and Jing Lin. Machinery health prognostics: A systematic review from data acquisition to RUL prediction. *Mechanical Systems and Signal Processing*, 104:799–834, 2018. URL: https://www.sciencedirect.com/science/article/pii/S0888327017305988, doi:10.1016/j.ymssp.2017.11.016.

[7] Xiang Li, Qian Ding, and Jian-Qiao Sun. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety*, 172:1–11, 2018. doi:10.1016/j.ress.2017.11.021.

[8] NASA. CMAPSS jet engine simulated data. NASA Open Data Portal. Accessed 2026-09-20; project uses FD001. URL: https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data.

[9] Emmanuel Ramasso and Abhinav Saxena. Review and analysis of algorithmic approaches developed for prognostics on CMAPSS dataset. In *Annual Conference of the PHM Society*, volume 6, 2014. Issue 1. URL: https://papers.phmsociety.org/index.php/phmconf/article/view/2512, doi:10.36001/phmconf.2014.v6i1.2512.

[10] Abhinav Saxena, Jose Celaya, Bhaskar Saha, Sankalita Saha, and Kai Goebel. Metrics for offline evaluation of prognostic performance. *International Journal of Prognostics and Health Management*, 1(1), 2010. URL: https://papers.phmsociety.org/index.php/ijphm/article/view/1336, doi:10.36001/ijphm.2010.v1i1.1336.

[11] Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management*, pages 1–9. IEEE, 2008. doi:10.1109/PHM.2008.4711414.

[12] scikit-learn developers. Cross-validation iterators for grouped data. User Guide. Accessed 2026-09-22; implementation version not selected. URL: https://scikit-learn.org/stable/modules/cross_validation.html#cross-validation-iterators-for-grouped-data.

[13] scikit-learn developers. Decomposing signals in components: Principal Component Analysis. User Guide. Accessed 2026-09-22; implementation version not selected. URL: https://scikit-learn.org/stable/modules/decomposition.html#pca.

[14] scikit-learn developers. HistGradientBoostingRegressor api. scikit-learn 1.9.1 documentation. Accessed 2026-09-27; implementation and documentation version 1.9.1. URL: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingRegressor.html.

[15] scikit-learn developers. Nested versus non-nested cross-validation. scikit-learn 1.9.1 documentation. Accessed 2026-09-27; documentation version 1.9.1. URL: https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html.

[16] SciPy developers. scipy.stats.bootstrap. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.bootstrap.html.

[17] SciPy developers. scipy.stats.false_discovery_control. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.false_discovery_control.html.

[18] SciPy developers. scipy.stats.wilcoxon. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.wilcoxon.html.

[19] statsmodels developers. statsmodels.tsa.stattools.adfuller. Online documentation. Accessed 2026-09-22; implementation version not selected. URL: https://www.statsmodels.org/stable/generated/statsmodels.tsa.stattools.adfuller.html.

[20] Tianyi Wang, Jianbo Yu, David Siegel, and Jay Lee. A similarity-based prognostics approach for remaining useful life estimation of engineered systems. In *2008 International Conference on Prognostics and Health Management*, 2008. doi:10.1109/PHM.2008.4711421.

[21] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi:10.2307/3001968.

[22] Shuai Zheng, Kosta Ristovski, Ahmed Farahat, and Chetan Gupta. Long short-term memory network for remaining useful life estimation. In *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)*, pages 88–95. IEEE, 2017. doi:10.1109/ICPHM.2017.7998311.